

Sunrise activity and mission systems handbook

General architecture, protocol, reconstruction, and debugging reference

Document date: 2026-08-22

Evidence date: 2026-08-22

Project: Sunrise

Primary branch during this investigation: `red-war-gameplay-host`

Observed source revision: `1ea4cb7`

1. Document status

This document describes the general Sunrise activity and mission system.

It is not a specification from Bungie.

It combines current source analysis, runtime captures, controlled tests, and reverse engineering.

The source worktree had local changes when this document was written. Therefore, the named revision does not describe every observed source file.

The installed DLL can also differ from the current source. Always compare the source hash and DLL hash before a runtime test.

This handbook does not replace the case records for Homecoming or Omega. Those records preserve detailed evidence for their missions.

This handbook uses Homecoming and Omega only as examples of general system behavior.

The language follows the requested ASD-STE100 skill where practical. It uses controlled terms, short instructions, and direct sentences.

This document is not a certified ASD-STE100 document. The skill does not include the official ASD-STE100 approved-word dictionary.

2. Purpose

Use this handbook to do these tasks:

1. Understand the complete client-to-mission system.
2. Identify the layer that owns a failure.
3. Distinguish a transport success from a mission success.
4. Reconstruct one mission without adding unrelated changes.
5. Design a bounded runtime test.
6. Preserve evidence for another investigator.
7. Avoid experiments that the existing evidence disproved.

The primary goal is faithful mission reconstruction.

Faithful reconstruction means that Sunrise supplies the native client with the state that the retail service supplied.

The native client should then use its own world, activity, scene, actor, effect, and presentation systems.

A local client hook is a secondary method. Use a hook when the host route is unavailable or when a pinned build needs a bridge.

3. Scope

This handbook covers these systems:

- Client bootstrap and platform emulation

- BAP service routing
- Account and session state
- Activity destination selection
- Activity-host allocation
- Matchmaking and advertisement data
- NAT and QoS handshakes
- Gameplay association and transport
- Group sessions and membership
- Activity messages
- Bubble authority and entity slots
- Scenario and roster extraction
- Spawn-set selection
- Auth and sense publication
- Mission director and activity script objects
- Sensors and trigger edges
- Scene and actor construction
- Model and presentation construction
- Visual effects
- World devices and collision
- Mission completion and progression
- Logging, capture, and test methods

This handbook does not claim that Sunrise has a complete implementation of these systems.

Section 53 identifies the current implementation boundaries and known gaps.

3.1 Navigation

Use these section groups:

- Sections 4 through 10: evidence, architecture, identity, project structure, and BAP bootstrap
- Sections 11 through 18: destination, sessions, world selection, matchmaking, QoS, and gameplay transport
- Sections 19 through 25: group session, parameters, activity messages, membership, slots, and bubble authority
- Sections 26 through 31: build data, roster publication,

- serialization, object schemas, and mission seed
- Sections 32 through 41: mission control, sensors, scenes, actors, VFX, world devices, progression, and AI
- Sections 42 through 48: state design, observability, recorders, reverse engineering, deployment, and diagnosis
- Sections 49 through 53: failed patterns, reconstruction playbook, case lessons, and implementation boundaries
- Sections 54 through 61: source map, settings, handoff rules, completion levels, glossary, log queries, and non-regression

4. Evidence labels

Every important conclusion must use one of these labels.

4.1 CONFIRMED

Direct source, a decoded packet, a log, or a controlled visual test proves the statement.

Example:

CONFIRMED: Service 6 returns service 7 in the current route table.

4.2 CURRENT SOURCE

The current worktree implements the statement.

This label does not prove that the installed DLL contains the implementation.

Example:

CURRENT SOURCE: A session-search request returns an empty present result.

4.3 HISTORICAL

An earlier branch, build, or runtime capture validated the statement.

This label does not prove that the current branch still implements the behavior.

Example:

HISTORICAL: A descriptor-bearing search result used a nested field-3 result shape.

4.4 INFERRED

The available evidence supports the statement, but no direct observation proves it.

The inference must identify its evidence.

Example:

INFERRED: Type 35 is a mission-director object because its state changes at mission-control boundaries.

4.5 HYPOTHESIS

The statement is a testable proposal.

Do not write a hypothesis as a fact.

Example:

HYPOTHESIS: The closed wall needs a registry that the destination does not mount.

4.6 DISPROVEN

A controlled test produced the opposite of the predicted result.

Do not repeat a disproven route without new evidence.

Example:

DISPROVEN: Changing the captured root transform moved the final rendered beam.

4.7 UNKNOWN

The evidence does not identify the behavior.

Unknown is a valid result. It defines the next research boundary.

5. Core system statement

Destiny 2 is an online world client.

The retail client contains maps, models, animations, effects, and many native systems.

The retail service supplies session state, authority, membership, mission state, and event inputs.

Sunrise must reproduce enough of that service state for the native client to continue.

Loading a package does not create a mission.

Loading a map does not create mission authority.

Creating a session does not create an actor.

Creating an actor does not create its effects.

Playing dialogue does not prove that a scene actor exists.

Receiving a valid packet does not prove that the client applied its payload.

Applying a payload does not prove that a renderer used the changed value.

This separation is the most important rule in this handbook.

6. End-to-end layer model

The mission path has these general layers:

1. Platform bootstrap
2. BAP connection and account services
3. Activity destination selection
4. Activity-session allocation
5. World, bubble, slice, and spawn selection
6. Public matchmaking or private activity-host selection
7. Gameplay peer and group session
8. Activity membership and entity-slot allocation
9. Bubble authority
10. Roster group registration
11. Runtime authority-object construction
12. Auth and sense body application
13. Mission director and activity script
14. Sensors and trigger inputs
15. Scene authority and selector state
16. Actors, models, animations, and authored events
17. Visual effects and renderer state
18. Mission completion and progression

Each layer can succeed while the next layer fails.

For example, a correct destination can load the world while the mission roster remains empty.

A correct roster can register a group while the runtime object factory rejects its object.

A runtime object can exist while its auth body has an invalid patch epoch.

A scene can play dialogue while its cast factory creates no actors.

An effect object can accept a coordinate write while the renderer uses another composite buffer.

6.1 Layer proof rule

Use this rule during every investigation:

Proof at layer N is not proof at layer N plus 1.

Record the first layer that fails.

Do not change a lower layer after it has stable proof unless new evidence invalidates that proof.

6.2 The two execution paths

Sunrise can use two broad execution paths.

Host simulation path

Sunrise publishes session, authority, script, and scene state.

The native client creates and updates the mission objects.

Use this path first when the protocol and schemas are available.

This path preserves native lifecycle behavior.

Client executor path

A local hook calls a native client function or changes native runtime state.

Use this path for diagnosis, for a pinned-build bridge, or when the host path cannot express the required operation.

This path is more dependent on the executable build.

It can also bypass lifecycle work that the host path normally causes.

Therefore, a direct native call can create a visible object without creating its authority, collision, AI, or completion state.

7. Success definitions

Define success before a test.

Use separate success levels.

7.1 Transport success

The client and server exchange a valid frame or packet.

7.2 Decode success

The target parser accepts the payload.

7.3 State success

The target manager stores the expected state.

7.4 Construction success

The target factory creates the expected runtime object.

7.5 Presentation success

The client shows the expected model, animation, sound, or effect.

7.6 Gameplay success

The object has the expected collision, trigger, AI, damage, or objective behavior.

7.7 Lifecycle success

The object starts, transitions, retires, and releases resources at the correct boundaries.

7.8 Mission success

The mission accepts the state transition and permits normal progression.

A visual match is not sufficient for mission success.

A mission can also progress with an incorrect visual result.

Record both results.

8. Identity domains

The system uses several identity domains.

Do not treat two identifiers as equal because their values look similar.

8.1 Account handle

The BAP activity-message envelope carries an account handle.

It identifies the client account in that service path.

8.2 SOID

SOID is the service object identifier used by account and character data.

Some records divide it into an account half and another identifier component.

8.3 Activity-session identifier

Service 7 gives the client a process-local activity-session identifier.

Zero means absent.

The current allocator starts at one.

8.4 Activity-host identifier

The activity-host identifier selects an activity-host endpoint.

The group parameter named `activityHost` must carry a valid nonzero identifier.

8.5 Group-session identifier

The gameplay group session has its own identifier.

It is not the BAP activity-session identifier.

8.6 Machine identifier

Gameplay membership and join descriptors carry a machine identity.

The local and ambassador machine identities can select different native join paths.

8.7 Member key

Activity membership uses a member key.

The key must agree with the descriptor and the joined session state.

8.8 Online-session identifier

In citizen advertisements, the online-session identifier is the ambassador activity-host identifier.

It is not the session identifier inside the descriptor.

8.9 Advertisement identifier

Matchmaking allocates an advertisement identifier.

It identifies one stored advertisement record.

8.10 Object reference

Auth and sense messages identify an object by a biased type and a biased index.

The wire value for the type is the slot type plus one.

The wire value for the index is the slot index plus 32768.

Do not use zero-filled values for these fields.

8.11 Scene and effect handles

Runtime scene and effect handles include a generation component.

An index alone is not a stable identity.

Log the full handle, generation, definition, scene handle, and time.

9. Project architecture

The source tree divides responsibilities by layer.

9.1 Sunrise/src/client

This directory contains client hooks, package readers, content extraction, and in-process retail-client integration.

Client code observes or changes native state in the installed executable.

This code is usually build-specific.

9.2 Sunrise/src/middleware

This directory contains wire schemas, bit codecs, protobuf codecs, and gameplay protocols.

Middleware code should not own long-lived process state.

It parses borrowed input and writes caller-owned output.

9.3 Sunrise/src/server

This directory contains the embedded BAP server, gameplay transport, peer sessions, group sessions, and web endpoints.

Server routes coordinate codecs and process state.

9.4 Sunrise/src/state

This directory contains process-owned transactional state.

It stores activity sessions, matchmaking records, group-host mappings, inventory, and build-data catalogs.

Do not store a borrowed packet span in this layer.

9.5 Sunrise/src/steam

This directory contains Steam interface emulation.

It supplies the platform behavior that the retail client expects.

9.6 Sunrise/src/core

This directory contains settings, logging, runtime controls, and user-interface support.

9.7 Ownership rule

Keep each state item in one ownership domain.

Use `State` for process-owned durable runtime data.

Use a connection object for connection-owned publication state.

Use a request or scratch object for borrowed input and temporary output.

Use a client hook only for native client state.

This rule prevents stale spans, split revisions, and partial commits.

10. Platform and bootstrap layer

The client must finish its platform and secure-channel bootstrap before mission services can operate.

This layer includes Steam emulation, sign-on exchange, relay registration, account translation, and basic configuration.

The exact service exchange can change between client builds.

The current BAP frame layer supports these outer frame types:

- Type 0: plaintext client request marker
- Type 1: authenticated AES-GCM frame
- Type 2: plaintext bootstrap request or response marker

The BAP parser separates the outer frame from the inner service request.

The inner request carries a service identifier, task identifier, and body.

The response must use the matching task identifier.

10.1 Pending-request rule

A request service that has a response service must receive a response.

If Sunrise stays silent, the client can keep the task in its pending ring.

Enough unanswered requests can stop later service work.

An unsupported route should return a schema-valid neutral response when the client expects a response.

One-way notification services are different. Do not invent a response for a one-way service.

10.2 Current BAP request services

CURRENT SOURCE: The frame registry defines these request services:

- 6: activity-host manager
- 8: activity message
- 10: web service
- 110: server-role web service
- 12: subscribe family
- 14: unsubscribe family
- 16: activity host
- 18: client configuration
- 21: purchased offers
- 23: account translation

- 25: server hello
- 29: one-way notification
- 30: start
- 32: user message
- 34: skill
- 36: unnamed request
- 38: unnamed request
- 40: unnamed request
- 42: matchmaking
- 44: clan
- 48: unnamed request
- 121: register subscriber
- 171: large one-way notification
- 250: echo
- 302: relay registration
- 304: Steam certificate signing
- 306: account from membership

10.3 Current BAP response services

CURRENT SOURCE: The matching response services are:

- 7: activity-host manager
- 11: web service
- 112: server-role web service
- 13: subscribe family
- 15: unsubscribe family
- 17: activity host
- 19: client configuration
- 22: purchased offers
- 24: account translation
- 26: server hello
- 31: start
- 33: user message
- 35: skill
- 37: response to service 36
- 39: response to service 38
- 41: response to service 40
- 43: matchmaking

- 45: clan
- 49: response to service 48
- 122: subscriber registration
- 251: echo
- 303: relay registration
- 305: Steam certificate signing
- 307: account from membership

10.4 Current server notifications

CURRENT SOURCE: Sunrise sends these uncorrelated notification services:

- 9: activity message
- 123: queue update

A notification header does not contain a response status field.

10.5 Staged service outcomes

CURRENT SOURCE: BAP routes use a staged service outcome for state changes.

The route prepares the response and the pending mutation first.

The server copies and encodes the response before it commits the mutation.

The commit validates the expected state revision again.

This order prevents state from advancing when the server cannot produce the response.

Use this sequence for all important service mutations:

1. Parse the complete request.
2. Validate all lengths and identifiers.
3. Prepare the output.
4. Prepare the state mutation.

5. Encode or copy the output.
6. Validate the pending mutation again.
7. Commit the mutation.
8. Publish later notifications from committed state.

Do not commit while a response still depends on fallible encoding.

10.6 Sensitive data rule

Some request bodies contain tokens, descriptors, or account data.

Keep sensitive input as a borrowed span during the request.

Do not copy it into global state unless the protocol requires durable storage.

Do not write raw sensitive payloads to ordinary logs.

Use a private analysis directory for an approved opaque capture.

11. Activity destination selection

The client sends its activity selection to BAP service 6.

Sunrise returns the allocated activity-session identifier on service 7.

This exchange selects a destination. It does not create mission objects.

11.1 Service 6 request

CONFIRMED: The full service 6 request used by the current parser is 7,719 bytes.

The request contains two selection copies.

The route selects the primary copy when the primary package is known.

It selects the secondary copy when the primary package is unknown and the secondary package is known.

The selected record can contain these values:

- Package name
- Selection reason
- Previous activity index
- Current activity index
- Activity element
- Arrival bubble hash
- Spawn-set hash
- Exact raw selection descriptor

The raw descriptor contains fields that Sunrise does not fully decode.

Preserve the exact descriptor bits when they are available.

Do not rebuild the descriptor from decoded scalar fields.

A rebuilt descriptor can lose unknown fields that the client later reads.

11.2 Destination validation

CURRENT SOURCE: The destination state uses these limits:

- Package name length: 40 characters
- Selection reason: minus 1 through 14
- Activity index: absent, zero, or 1 through 4,094
- Activity element: absent or zero through 510
- Raw descriptor storage: 232 bytes
- Raw descriptor length: at most 1,814 bits
- Common captured descriptor length: 372 bits
- Absent spawn-set hash: `0x811C9DC5`

Reject a value that exceeds its storage or wire range.

Do not truncate a destination name or bit count silently.

11.3 Arrival override

An authored arrival override can change the selected arrival once.

Use a one-shot override for a controlled mission entry.

Do not apply it to every later destination selection.

A persistent arrival override can break transitions inside the mission.

11.4 Forced destination

The operator can select a process-local forced destination.

The forced destination applies after the client selection and the authored arrival override.

A forced world destination requires these values:

- Package
- Bubble
- Slice

A cinematic destination can be bubbleless.

The forced bubble range is zero through 63.

The forced slice range is zero through 1,022.

The spawn set is optional.

The override can replace the package name inside the preserved descriptor.

This operation keeps the unknown descriptor bits intact.

Forced destinations are not saved to durable configuration state.

11.5 Service 7 response

CONFIRMED: The minimal service 7 body is 137 bytes.

It has this content:

1. A discriminator with value 2
2. One big-endian 64-bit activity-session identifier
3. 128 zero bytes for minimal activity data

The activity-session identifier must not be zero.

The zero activity-data block is sufficient for this response path.

It does not describe the complete runtime mission.

11.6 Failure interpretation

Use these results to locate a destination failure:

- No service 6 parse: bootstrap or framing failure
- Service 6 parse but no service 7: route or response failure
- Service 7 with zero identifier: invalid allocation
- Correct package but wrong world: destination or package mapping failure
- Correct world but wrong arrival: bubble, slice, or spawn failure
- Correct arrival but no mission: continue at roster and mission authority

Do not change the BAP bootstrap when the correct world already loads.

11.7 Wire and runtime selection records

The BAP wire descriptor and the native runtime selection record are different representations.

The BAP parser reads a variable-length bit descriptor.

The native client later constructs an internal selection record for its manager path.

HISTORICAL: The observed native runtime record was 0x1B0 bytes.

HISTORICAL: Its route byte was at offset 0x12.

Zero selected the local route in the observed build.

A nonzero value selected the authored route.

These offsets and values are build-specific.

Do not write a wire-field result directly into a runtime record without its native construction lifecycle.

11.8 Recovered wire descriptor model

HISTORICAL: The recovered wire descriptor used these ordered fields:

- Four-bit selection reason
- 12-bit source activity
- 12-bit destination activity
- Optional nine-bit element index
- Five-bit skull count and repeated skull entries
- Eight-bit unknown value
- Optional peer byte
- Optional 32-bit arrival-bubble hash
- Optional 32-bit spawn-set hash
- Forty biased signed package-name bytes
- Trailing Boolean
- First counted tail with 96-bit entries
- Second counted tail with 13-bit entries

The minimum common form used 372 bits.

Observed authored forms used 620 bits and 716 bits.

One 716-bit form contained one 96-bit reference entry.

One 620-bit form contained no reference entry.

Therefore, descriptor size alone does not identify tail contents.

11.9 Dynamic descriptor values

HISTORICAL: One recovered sensitive scalar matched the configured primary SOID.

Another scalar matched a per-activity fireteam or activity nonce.

The nonce changed on each activity setup.

Do not cache one authored descriptor and assume that its nonce remains valid.

Preserve the current client's exact descriptor when possible.

11.10 Descriptor timing

In one historical Homecoming route, service 6 occurred after the client committed its runtime route.

Changing the service-6 descriptor therefore did not change that already committed route.

A separate test delivered a larger descriptor in periodic global-state messages.

The client still kept its local runtime route in that test.

The result showed that delivery to one wire consumer did not prove ingestion by the next-load runtime producer.

General rule:

Identify the producer of the exact runtime record that the decision reads.

12. Activity-session state

An activity session binds a destination to one joined client and its activity

state.

It owns the state that later service 9 messages publish.

12.1 Session capacity

CURRENT SOURCE: The activity-session table has 16 records.

The table must hold a private session and activity-host sessions for several regions.

A full table evicts its oldest record.

The extra capacity reduces the risk that regional hosts evict the private session.

12.2 Identifier and revision rules

Zero is the absent session identifier.

Session allocation starts at one.

State revisions and allocator revisions start at one.

Revisions do not wrap.

A cleared pending transaction therefore cannot match a live revision.

12.3 Session record

CURRENT SOURCE: One session record owns these values:

- Committed destination
- Client-held entity-slot mask
- Server-reserved entity-slot mask
- Membership state
- Bubble-authority state
- Session identifier

- Bound member key
- Creation revision
- Record revision
- Joined revision
- Occupied flag
- Joined flag

The client-held and server-reserved masks must be disjoint.

A returned client slot becomes free. It does not become a server-reserved slot.

12.4 Pending allocation

The server prepares an allocation before it acquires the state write lock.

The pending allocation records these expected values:

- Destination
- New session identifier
- Account identifier base
- State revision
- Allocator revision
- Next session identifier
- Target table slot

The commit compares the expected values with current state.

It rejects the allocation if another mutation changed them.

This design prevents two requests from committing the same identifier or slot.

12.5 Group-host session state

The group-host session table is separate from the activity-session table.

CURRENT SOURCE: It has eight records.

It maps gameplay region identities to activity-host sessions.

Do not use the activity-session table as a replacement for this mapping.

Migration can require two group identities to refer to related host state.

13. World, bubble, slice, and spawn selection

The destination selects a world context.

The bubble and slice select a region state inside that context.

The spawn set selects an authored arrival point.

13.1 Bubble

A bubble is a scenario region.

The global activity state contains 64 bubble-state bytes.

Usable bubble indices are zero through 63.

13.2 Slice

A bubble can have as many as eight slice states in the current scenario catalog.

A slice-set index maps to its bubble with this operation:

```
bubble = sliceSetIndex >> 3
```

The global activity state publishes the selected initial slice.

Do not replace an explicit forced slice with a later derived value.

13.3 Arrival bubble and current region

The destination arrival bubble selects the initial arrival.

The membership-reported region identifies the player's current region after movement.

These values have different purposes.

The current player location can therefore differ from the destination arrival.

The spawn override continues to use the destination arrival.

13.4 Spawn set

A spawn-set name uses an FNV-1 hash.

The spawn-set catalog associates each point with a map stem and package availability.

A known spawn hash is not sufficient.

The destination must load a package that declares the spawn set.

If the row is known but its package is not loaded, Sunrise uses the absent spawn hash.

An absent catalog row is not always proof that the source content lacks the spawn.

The catalog can reach a capacity limit before it records all rows.

13.5 Current spawn catalog limits

CURRENT SOURCE: The spawn catalog has these limits:

- 65,536 spawn points
- 128 map stems
- 4,096 distinct name hashes
- 32 bytes for one bubble mask
- 12 package references per spawn set

The observed catalog contained these live counts during this investigation:

- 77 map stems
- 2,090 distinct spawn hashes

These counts depend on the extracted build data.

13.6 World phase

CURRENT SOURCE: Sunrise tracks three broad world phases:

- Idle
- Transitioning
- Arrived

The authored mission seed arms only after the client reaches `Arrived`.

It clears when the client returns to `Idle`.

This timing prevents mission objects from filtering the first player citizen during phase zero.

13.7 Spawn hold

The client polls a native spawn gate during world loading.

Sunrise can hold the spawn while the world loader or activity loader is active.

Release the spawn only after the world reaches `Arrived`.

Do not release it when the native gate first reports permission.

An early spawn can run before the fade channel is armed.

The client can then remain on a black screen because the gate does not poll again.

CURRENT SOURCE: The default hold timeout is 30 seconds.

CURRENT SOURCE: The maximum configured hold timeout is 600 seconds.

14. Activity-host endpoint

Service 16 resolves one activity-host identifier to a relay endpoint.

Service 17 returns the endpoint.

Do not confuse this exchange with service 6 and service 7.

Service 6 allocates an activity session.

Service 16 resolves an activity-host endpoint.

14.1 Service 16 request

CONFIRMED: The request body is exactly eight bytes.

It contains one big-endian 64-bit activity-host identifier.

14.2 Service 17 response

CONFIRMED: The response body is exactly 16 bytes.

It contains these fields:

1. The echoed 64-bit activity-host identifier
2. A big-endian 32-bit relay IPv4 address
3. A neutral 16-bit value
4. A 16-bit relay port

The address and port must be nonzero.

The echoed identifier must equal the requested identifier.

The client rejects a mismatched identifier.

Silence can leave the request in the client pending ring.

15. Private and public activity routes

The client can enter an activity through a private path or a public citizen path.

These paths share world content but use different membership and matchmaking work.

15.1 Private route

A forced destination can load a region without public matchmaking.

This route is useful for world and content tests.

It does not prove that the public mission host path works.

15.2 Public route

The public route uses a region activity host, an advertisement, and a citizen membership row.

The ambassador slot must differ from the local slot for the observed citizen path.

CURRENT SOURCE: When the local slot is zero, the advertisement uses slot one as ambassador.

When the local slot is not zero, it uses slot zero.

The advertisement host identifier must match the `activityHost` group parameter.

A mismatch causes the client to report `public_activity_host_mismatch`.

15.3 Homecoming route lesson

HISTORICAL: Homecoming first stayed on a private or fireteam path.

Changing the region transition input to public started the citizen and search path.

The change occurred at the transition-starter call.

Forcing a global reader or an orbit destination did not identify the correct boundary.

General rule:

Change the input at the exact native decision point. Do not force a downstream result globally.

15.4 Advertisement readiness

The advertisement can have these states:

- Ready: a host session and its descriptor are available
- Pending: the slot is claimed but the host session is not available
- Absent: no valid advertisement source is available

Do not allocate a missing host session inside a staged service 9 push.

Host allocation changes the global state revision.

Perform allocation in a separate service slice.

Publish only committed state.

16. Matchmaking

BAP service 42 carries matchmaking requests.

BAP service 43 returns request-specific results.

The body uses protobuf fields.

16.1 Request kinds

CURRENT SOURCE: The request selector has these values:

- 0: none or invalid
- 1: session search
- 2: advertisement update
- 3: advertisement delete
- 4: configuration
- 5: rejoin advertisement update
- 6: rejoin advertisement delete
- 7: locate session
- 8: live statistics

16.2 Join descriptor

A join descriptor is exactly 128 opaque bytes.

Treat it as an exact byte object unless its complete schema is known.

Do not change unknown bytes.

Do not log a raw descriptor in an ordinary log.

16.3 Matchmaking state

CURRENT SOURCE: The matchmaking state has four contexts.

Each context has 16 variant records.

Generated identifiers start at one.

Zero means absent.

State mutations use transaction guards.

16.4 Current response behavior

CURRENT SOURCE: Session search returns a present but empty field-3 result.

CURRENT SOURCE: Configuration returns lane and bubble policies.

The lane policy uses these values:

- Service configuration: 1
- Search-only interval: 60 seconds
- Desperation interval: 60 seconds
- One present provider-policy entry

The bubble policy uses these values:

- Maximum players: 9
- Maximum posse size: 3
- Maximum matchmade players: 6
- Service configuration: 1

CURRENT SOURCE: Advertisement update returns a root result and a nested advertisement identifier.

CURRENT SOURCE: Locate session can return a field-7 result with an identifier and a descriptor wrapper.

The maximum descriptor-bearing locate response is 148 bytes.

16.5 Historical search result

HISTORICAL: A working descriptor-bearing search result used this protobuf structure:

```
root field 3
  repeated field 1
    field 1 descriptor message
      field 1 exact 128-byte descriptor
```

A result with one incorrect nesting level caused a policy-31 fatal decode.

Do not describe this historical search result as current branch behavior.

16.6 Configuration presence

An empty but present configuration is not neutral.

HISTORICAL: Zero timing thresholds caused an immediate advertisement change.

That change also changed the generation and cancelled the active search.

Use valid nonzero policy values when the field is present.

Omit an optional field when no valid meaning is known.

16.7 Matchmaking proof boundary

A valid search or advertisement proves only matchmaking state.

It does not prove that a gameplay association exists.

It does not prove that the group session joined.

It does not prove that the mission manager became active.

17. NAT introduction and QoS

The gameplay path can require NAT introduction and QoS suitability checks.

These messages are outside the BAP activity-message envelope.

17.1 NAT introduction

HISTORICAL: NAT introduction request type 13 received reply type 12.

Use the captured endpoint and association context when you validate this exchange.

17.2 QoS request

HISTORICAL: The Demonware QoS request used type 0x28 and an 18-byte header.

The request carried these observed fields:

- Byte 0: type
- Bytes 1 through 8: combined timestamp and payload-size value
- Bytes 5 through 8: requested payload size
- Bytes 9 through 12: probe key
- Bytes 13 through 16: session identifier
- Byte 17: trailing request byte

17.3 QoS response

HISTORICAL: The response used type 0x29.

The header contained these observed fields:

- Byte 0: type
- Bytes 1 through 4: echoed probe key
- Bytes 5 through 12: echoed request bytes 1 through 8
- Byte 13: acceptance value 1
- Bytes 14 through 17: little-endian payload length
- Bytes 18 through 21: zero
- Bytes 22 through 25: zero
- Byte 26: zero
- Byte 27 onward: structured payload

A correct header cleared the invalid-identifier, refusal, and empty-response errors.

A zero payload still failed the decoder and suitability test.

The full payload structure was not recovered.

VMProtect protected the relevant runtime decoder.

UNKNOWN: The complete general QoS payload schema remains unresolved.

Do not claim that the current branch implements the complete historical response.

18. Gameplay network layers

The gameplay connection has several nested layers.

Use this order when you diagnose it:

1. UDP endpoint
2. Association
3. DTLS or protected transport
4. Peer transport
5. Group session
6. Group parameters and membership
7. Activity-host publication

18.1 Current local topology

CURRENT SOURCE: The embedded topology uses loopback by default.

The bind address, advertised address, and transport address are `127.0.0.1`.

The default gameplay port is 30976.

Use an even gameplay port.

Do not use discovery ports 3074 or 3075 for the gameplay endpoint.

18.2 Association lifetime

A peer can reconnect shortly after a connect-close message.

The observed delay can be approximately 50 milliseconds.

Do not release the group session after an ordinary connect close.

Release it after the association timeout or an explicit terminal lifecycle event.

18.3 Peer connect messages

The peer-connect registry uses these known message identifiers:

- 5: connect request
- 6: connect response
- 7: connect refusal
- 8: establish
- 9: closed
- 10: join request
- 14: join refusal

18.4 Established and out-of-band packets

The first-bit grammar distinguishes out-of-band packets from established peer packets.

Parse this discriminator before you apply the established packet schema.

18.5 Application-ready boundary

The establishment exchange alone does not make the peer application-ready.

The client must also exchange a normal connected packet.

Before that boundary, the transport can acknowledge reliable records without dispatching their group messages.

Wait for application-ready before you send important membership or parameter records.

18.6 Reliable publication

Reliable records use the group-session identifier as part of their queue key.

The queue can fill while join promotion and activity-host parameter messages retry.

Preserve sequencing, acknowledgement state, reliable assembly, and the view signature.

Do not interpret an acknowledgement as proof that the application dispatched the record.

18.7 NetAddr capture

Capture the client `NetAddr` from the gameplay transport.

Echo the exact address form in membership where the protocol requires it.

A byte-level mismatch can prevent self-member resolution or host handoff.

18.8 View signature

The group view has a signature.

A signature mismatch can block entity output after the group appears connected.

Validate the signature before you investigate mission-object schemas.

19. Gameplay group session

The group session creates the gameplay peer group.

It publishes membership, host identity, parameters, and transitions.

19.1 Known session messages

The current codec registry names these group-session messages:

- 11: peer connect
- 12: join complete
- 13: join abort
- 15: leave session
- 16: leave acknowledgement
- 17: disband
- 18: boot
- 19: host handoff
- 21: host transition
- 22: host reestablish
- 25: peer reestablish
- 26: peer establish
- 29: time synchronization
- 30: membership update

19.2 Known decoded sizes

The recovered decoders use these body sizes in the same message order:

- Peer connect: 24 bytes
- Join complete: 24 bytes
- Join abort: 24 bytes
- Leave session: 8 bytes
- Leave acknowledgement: 8 bytes
- Disband: 24 bytes

- Boot: 24 bytes
- Host handoff: 100 bytes
- Host transition: 16 bytes
- Host reestablish: 136 bytes
- Peer reestablish: 8 bytes
- Peer establish: 8 bytes
- Time synchronization: 48 bytes
- Membership update: 31,104 bytes

Treat these values as build-specific until another executable confirms them.

19.3 Membership capacity

The group membership snapshot supports 32 members and 32 players.

The snapshot is complete, not a sparse patch.

The client clears and rebuilds the table when it accepts a newer snapshot.

The membership revision starts at one.

A later snapshot must have a strictly greater revision.

19.4 Member state ladder

The observed member states use this progression:

- 3: reserved ambassador
- 5: connected
- 7: joined
- 8: waiting
- 9: ready
- 10: established

Do not skip a required state edge only because the final numeric state is known.

Native managers can perform side effects on each edge.

The local row must reach the established state for normal entity publication.

19.5 Self-member resolution

The client finds its own membership row by `NetAddr` and join identity.

The own row must byte-match the expected address form.

The join identifier must also match the active join operation.

An incorrect row can leave the group connected at transport level but absent at member level.

19.6 Connection-present state

The membership row includes connection-present state.

This value drives peer resolution.

Do not mark a remote member present before the corresponding peer can resolve.

19.7 Membership hash

The membership message ends with a state-replica hash.

The hash must byte-match the complete encoded snapshot.

Do not calculate it over a partial row set.

19.8 Host reestablish body

The recovered 136-byte host-reestablish body contains these fields:

- Session identifier at offset 0
- Machine identifier at offset 8
- `NetAddr` at offset 16
- A 16-byte identity block at offset `0x66`
- An 18-byte identity block at offset `0x76`

The `NetAddr` method uses three bits.

Methods zero through five carry 41 address bytes.

Methods six and seven carry 85 address bytes.

HISTORICAL: Empty identity blocks let a manager receive an activation request but remain inactive.

This result shows that a valid transport message can still lack required semantic identity.

19.9 Host handoff

Host handoff is the known native message that arms the manager pending-member index.

The target address must byte-match the selected member address.

A handoff to a logically equal but byte-different address can fail.

19.10 Host transition

The host-transition body carries a count and one opaque 32-bit value.

Clamp the count to zero through 100.

Do not assign a meaning to the opaque value without direct evidence.

20. Group parameters and publication order

Group parameters publish shared session state.

The client uses parameter update message 38 and parameter request message 39.

20.1 Parameter registry

CURRENT SOURCE: The registry contains these indices:

- 0: worldControllerGoalData
- 1: activeJoinControls
- 2: atomicCascadeJoinData
- 3: activityHost
- 4: jipGate
- 5: activitySelection
- 6: activitySelectionResponses
- 7: currentActivity
- 8: previousActivity
- 9: userJoinControls
- 10: desiredJoinControls
- 11: language
- 12: requestedRemoteJoinData
- 13: remoteJoinData
- 14: remoteJoinResult
- 15: hostSelected
- 16: matchmakingMessaging
- 17: matchmakingAbortRequested
- 18: sessionDisbandReason
- 19: matchmakingProgress
- 20: initialSliceSetStatus
- 21: publicSessionReservations
- 22: matchmakingData
- 23: matchmakingPeerData
- 24: networkQuality

20.2 Minimum join publication

The peer needs a membership snapshot and at least one parameter

update.

The connection does not finish its application join from membership alone.

Publish these records after the peer becomes application-ready.

20.3 Activity-host parameter

The `activityHost` parameter must carry a nonzero activity-host identifier.

Do not publish a neutral zero value as a placeholder.

HISTORICAL: A zero identifier latched an unusable client state.

If no valid identifier exists, defer the parameter.

20.4 Publication order

Use this general order for a new group join:

1. Finish peer establishment.
2. Exchange one normal connected packet.
3. Publish a complete membership snapshot.
4. Publish required group parameters.
5. Promote the local member through valid states.
6. Publish activity-host messages.

The exact order can require a mission-specific or build-specific adjustment.

Keep each change small enough to identify the required edge.

20.5 Migration publication

A host migration can require this observed sequence:

1. Publish peer-reestablish membership.
2. Publish the migrated `activityHost` identifier.
3. Publish baseline host reestablish.
4. Publish host handoff.
5. Publish host transition.
6. Publish the return handoff when required.
7. Publish activation host reestablish.

Do not compress this sequence into one final-state write.

The native client can use the intermediate messages to create manager state.

20.6 Homecoming activation lesson

HISTORICAL: Homecoming reached the native manager activation function.

The function returned true.

The manager active flag still remained zero.

The activation identity blocks were empty.

Therefore, the function return did not prove an active authored manager.

General rule:

Validate the postcondition in the owning manager. Do not trust only the call return.

21. Activity-message protocol

BAP service 8 carries client-to-activity-host messages.

BAP notification service 9 carries server-to-client activity-host messages.

21.1 Envelope

CURRENT SOURCE: The activity-message envelope contains these fields:

1. Big-endian 64-bit account handle
2. One-byte discriminator with value 1
3. Big-endian 32-bit message type
4. Big-endian 32-bit payload length
5. Big-endian 32-bit peer-heard mask
6. Exact payload bytes

The maximum payload length is `0x7D800` bytes.

The declared length must equal the remaining body length.

Treat the request payload as sensitive borrowed input.

21.2 Server-to-client message types

The current implementation publishes these important message types:

- 0: entity-slot notification
- 1: global activity state
- 4: join result or pending-join notification
- 5: auth and sense update
- 12: membership replication
- 54: bubble-host table

The outbound registry permits message identifiers through 58.

21.3 Client-to-server message types

The current route recognizes these client message types:

- 3: join request
- 6: sense update
- 8: request activity host
- 11: start new activity

- 13: request peer reservation
- 14: release peer reservation
- 15: peer leave request
- 16: client keepalive
- 18: state refresh
- 19: incident report
- 20: entity-slot grant request
- 21: entity-slot return
- 22: client authoritative data
- 23: client identity
- 26: abandon authority
- 27: purge request
- 29: reset acknowledgement
- 31: per-bubble query answer
- 32: whole-activity query answer
- 33: abdicate authority
- 34: debug command
- 37: connectivity failure report
- 38: membership acknowledgement
- 39: client heartbeat
- 43: bug-claw report
- 46: lag-switch report
- 47: connection-quality report
- 48: speculative migration report
- 49: high-water report
- 50: inspirations refresh
- 52: patch epoch

Several reports are accepted as neutral no-operations in the current server.

An accepted no-operation prevents a client wait. It does not implement the reported feature.

21.4 Delivery modes

CURRENT SOURCE: A handled request can schedule these notification groups:

- No notification
- Join notifications
- Entity-slot notification
- Membership notification
- Refresh notifications
- Authoritative notifications

21.5 Mutation domains

CURRENT SOURCE: A request can mutate one of these state domains:

- None
- Entity slots
- Membership
- Patch epoch

Keep the mutation and its notification plan in one staged result.

21.6 State refresh

A state-refresh request publishes these messages in this order:

1. Global activity state
2. Membership
3. Activity roster

Preserve this order.

The roster body can refer to global state and membership data.

21.7 Authoritative delta

An authoritative update can also require a membership or roster refresh.

Send membership when host state changes.

Send the roster immediately when the client moves to a new region.

The new bubble has no usable authority until it receives the required roster and grant state.

21.8 Transition token

Membership mirrors a transition token.

A token change starts an activity load and increases the roster publication rate.

Do not change the token for an ordinary stable refresh.

21.9 Patch epoch

The client sends message type 52 with its patch epoch.

Store this epoch on the connection.

Do not store it as one global process value.

The auth and sense phase must echo the exact epoch for that connection.

An incorrect epoch can make the client skip the body phase without a clear decode error.

22. Global activity state

Message type 1 publishes the global activity state.

It describes the selected world state for the activity-host connection.

22.1 Size

CONFIRMED: The meaningful prefix uses at least 1,161 bits.

The encoded body therefore needs at least 146 bytes.

Do not round a bit count down to complete bytes.

22.2 Main contents

The current body includes these values:

- 64 bubble-state bytes
- Selected initial slice set
- Spawn-set hash
- Embedded activity-selection descriptor
- Activity state and transition values

Preserve the exact activity-selection descriptor where possible.

22.3 Global-state boundary

A correct type-1 body proves that the client knows the destination state.

It does not prove that roster objects exist.

Continue with type 12, type 54, and type 5 evidence.

23. Activity membership

Message type 12 publishes activity membership.

Activity membership is different from gameplay group membership.

The two systems must agree on identity and region.

23.1 Local snapshot size

CONFIRMED: The local snapshot uses 29,968 bits.

This value is 3,746 bytes.

23.2 Remote citizen cost

A remote citizen adds 1,081 membership bits.

Its descriptor adds 1,024 bits.

Keep the descriptor at exactly 128 bytes.

23.3 Stable epoch

Membership epoch zero preserves the existing peer table.

An epoch change clears the table.

Do not change the epoch during a stable refresh.

23.4 Local identity

The snapshot publishes the local player and member identity.

The activity member key must agree with the joined session record.

The player identity must also agree with the group-session row.

23.5 Citizen advertisement

A citizen advertisement appears in one region record.

The client adopts the citizen only for its pending region.

The advertisement has these important identity rules:

- The member key matches the descriptor machine identity.
- The `NetAddr` matches the gameplay peer endpoint.
- The online-session identifier is the ambassador activity-host

identifier.

- The ambassador slot differs from the local slot for the citizen path.
- The region index equals the target region.

23.6 Region identities

CURRENT SOURCE: Regional machine and online-session identities derive from one endpoint base identity.

The derivation uses this odd stride:

```
0x9E3779B97F4A7C15 * (region + 1)
```

The result is combined with the base identity by XOR.

The odd stride reduces simple collisions between adjacent regions.

23.7 Source-session destination

A regional activity host can inherit its destination from a source activity session.

This behavior keeps the region host in the same activity context.

Validate the source session before the allocation commits.

24. Entity slots

Entity slots divide runtime object authority between the client and server.

24.1 Slot space

CURRENT SOURCE: The activity has 8,192 entity slots.

One complete mask is 1,024 bytes.

24.2 Server reserve

CURRENT SOURCE: The default server reserve is 256 high slots.

The minimum configured reserve is eight slots.

The client requires at least 4,096 grantable slots in the observed topology.

A disabled gameplay topology reserves zero server slots.

24.3 Join allocation

The join grants low slots to the client.

It keeps the server reserve in high slots.

The masks must remain disjoint.

24.4 Slot lifecycle

The client requests slots with message type 20.

It returns slots with message type 21.

The server records the grant or return in the session transaction.

Publish type 0 after the mutation commits.

24.5 Slot proof boundary

A granted slot gives the client authority capacity.

It does not create a scenario roster object.

Roster construction uses authored slot descriptors and object references.

25. Bubble authority

Bubble authority controls which host can publish objects in each region.

25.1 Table size

The table has 65 entries.

Entries zero through 63 are usable bubbles.

Entry 64 is a first-send fallback.

Do not grant entry 64 as a normal bubble.

25.2 Grant tokens

Grant tokens start at one.

Zero means cleared.

Tokens belong to the bounded activity-session record.

They reset when the session record resets.

25.3 Region movement

A client that enters a new bubble needs a grant for that bubble.

Publish the new grant before or with the new region roster.

Do not assume that authority for the previous bubble applies to the next bubble.

25.4 Slice range

The destination wire can name slice values through 1,022.

The current bubble grant representation supports values through 511.

Validate this difference when a high slice index appears.

Do not truncate a high slice silently.

26. Build-data and content extraction

Sunrise reads installed client content to build scenario, roster, and spawn catalogs.

This layer discovers authored definitions.

It does not create native runtime objects.

26.1 Content proof levels

Use these separate statements:

- Package found
- Package loaded
- Scenario selected
- Registry discovered
- Registry mounted
- Roster group published
- Runtime object constructed
- Auth body applied
- Component active
- Renderer or gameplay system used the component

Never replace this list with the statement `the asset exists`.

26.2 Scenario catalog limits

CURRENT SOURCE: The scenario catalog has a capacity of 512 definitions.

The observed extraction contained 468 live definitions.

One scenario definition can contain these items:

- Package name and package tag
- 64 bubble records
- Top-level roster groups
- Bubble-local roster groups
- Group masks
- Spawn stem
- Bubble-state values
- Initial slice hash and index
- Map index
- Declared package list

26.3 Scenario names and bubbles

The maximum stored scenario name length is 40 characters.

A scenario stores 64 bubbles.

The enabled bubble-state byte is `0x80`.

The disabled bubble-state byte is `0x7F`.

26.4 Roster catalog limits

CURRENT SOURCE: The scenario catalog supports these roster limits:

- 128 roster groups
- 1,280 roster slots
- Four selected top-level groups
- Four selected bubble-local groups
- 32 characters for a spawn stem
- Eight declared destination packages

The observed extraction contained 68 live groups and 1,218 installed slots.

The highest observed slot type was 72.

These live counts are build-data observations. They are not protocol limits for every client build.

26.5 Roster group record

A roster group contains these important values:

- Registry key
- Object tag
- Slot count
- Slot types
- Slot flags
- Explicit authored slot indices

The explicit slot index is part of the object identity.

Do not use the slot ordinal as the authored index.

Some registries contain host-only records that do not have a descriptor-backed client slot.

Omit a host-only record from a descriptor-backed activity roster.

26.6 Staged scenario build

The build-data pass must tolerate package blocks that arrive in stages.

Use this build sequence:

1. Collect candidate scenario data.
2. Record unresolved tags.
3. Resolve tags after later blocks arrive.
4. Compact valid records.
5. Walk roster registries.
6. Publish the completed catalog.

An empty first pass is not proof that the scenario has no records.

The required block keys can arrive later.

Rearm the pass when unresolved keys remain.

26.7 Registry walk

For each scenario, inspect all bubbles and all slice states.

For each slice state, inspect its entries, registries, and the known registry descriptor arrays.

Do not stop after the first slice that contains the wanted key.

A later slice can omit or replace that key.

26.8 Safe top-level groups

A top-level group remains active across the complete activity.

Therefore, select top-level keys from the intersection of every slice set.

Drop a key that is absent from one slice set.

HISTORICAL: A key that disappeared during slice teardown could crash the client.

The intersection rule prevents publication of an object that cannot survive every selected slice.

26.9 Bubble-local groups

A bubble-local group applies only to one bubble or slice context.

Publish it inside the matching bubble block.

Do not promote it to a global group only because it contains a useful object.

26.10 Candidate ordering

The current candidate sort gives participation and lifetime groups early priority.

The selected top-level set must include a type-13 participation group.

It must also include type 17 across the selected top-level groups.

These objects establish essential player and activity lifetime state.

26.11 Authored exceptions

CURRENT SOURCE: The `rosterForceAuthored` setting adds exact Omega opening keys outside the global whitelist.

This rule is a case-specific exception.

It is not a universal roster-selection rule.

The setting is off by default.

Changing it rebuilds the content cache.

Document every keyed exception with its mission, purpose, and removal condition.

27. Auth and sense publication

Activity-message type 5 publishes roster groups and object state.

The protocol has two semantic phases.

27.1 Phase 1: group registration

Phase 1 registers these items:

- Group keys
- Slot descriptors
- Top-level or bubble-local ownership
- Bubble authority grants

This phase tells the client which authored objects can exist.

It does not apply the complete object bodies.

27.2 Phase 2: object bodies

Phase 2 sends auth and sense reset data for the registered objects.

This phase initializes or changes the runtime authority objects.

The body must use the patch epoch from the same connection.

27.3 Publication order

Publish all selected top-level groups before bubble-local groups.

The current message supports eight groups in total.

Do not exceed this group capacity.

27.4 Slot flags

The known slot flags are:

- Bit 0, value 1: sense data
- Bit 1, value 2: auth data

A slot can carry either flag or both flags.

Only encode a body that the descriptor flags permit.

27.5 Object-reference bias

Encode the object type as `type + 1`.

Encode the object index as `index + 32768`.

The authored index must not exceed 32,767.

A zero wire value does not mean type zero or index zero in this reference.

27.6 Exact remainder length

Each group and object block has an exact bit remainder.

A one-bit error can shift all later blocks.

The client can then read plausible but incorrect values without a clear outer decode error.

Calculate the complete body length before publication.

Verify the final writer position against that length.

27.7 First reset target

The first auth reset bit must target the live mirror that the client owns.

Do not target an inactive local copy only because its structure has the same schema.

27.8 State sequence

The type-5 `stateSequence` identifies the object generation.

It is not an ordinary packet revision.

The first three warmup publications can advance this value.

After warmup, keep it stable while the group set remains stable.

Changing the generation can tear down every roster-owned object.

Change it only when the roster group set changes or a true rebuild is required.

27.9 Runtime existence ladder

Use this ladder for every roster object:

1. Package contains a definition tag.
2. Destination selects the scenario.
3. The selected scenario exposes the registry group.
4. Phase 1 registers the group and slot.
5. The native factory constructs a runtime authority object.
6. Phase 2 applies the object body.
7. A native component becomes active.
8. A renderer or gameplay system consumes the component.

Record the first missing step.

Do not tune a phase-2 body when step 5 never occurs.

28. Serialization rules

Mission codecs mix byte-aligned fields and bit-aligned fields.

Use the schema for the exact message and executable build.

28.1 Bit count and byte count

Store a bit count as a bit count.

Calculate its byte storage with a ceiling division.

Use this expression:

```
bytes = (bits + 7) / 8
```

Do not use integer truncation.

28.2 Endianness

BAP scalar fields commonly use big-endian encoding.

Some lower protocol fields and opaque native bodies use little-endian encoding.

Do not apply one byte order to a complete message without a field-level schema.

28.3 Protobuf presence

In protobuf, an absent field and a present zero field can have different behavior.

Use an absent optional field when its valid meaning is unknown.

Use a present empty message only when the client expects message presence.

28.4 Exact opaque fields

Preserve these items as exact bytes unless their schemas are complete:

- Activity-selection descriptor
- Join descriptor
- Identity blocks
- Unknown manager payloads
- Native effect definitions

28.5 Biased values

Check every biased field before encoding.

Confirm that the unmodified value fits the permitted range.

Then apply the bias once.

Do not bias an already biased captured value.

28.6 Optional branches

An optional branch changes all later bit positions.

Record the presence bit and the branch length in a decoder trace.

Do not infer the branch from the final byte size alone.

28.7 Neutral body rule

A neutral body must still satisfy the complete schema.

Zero-filled storage is not a neutral body when the schema contains biased keys, required selectors, or time windows.

Build one explicit neutral encoder for each recovered type.

29. Core roster object types

The recovered object types below are important to current mission reconstruction.

Their meanings are based on source, native strings, and runtime behavior.

Do not assume that the same exact body applies to every resource of the same broad type.

29.1 Type 13: participation

CONFIRMED: The recovered participation body uses 192 base bits.

It adds 32 region bits when the region field is present.

The body binds the full character SOID, player key, and region.

Its biased identity fields cannot use a zero-filled placeholder.

This object connects the player citizen to activity participation.

29.2 Type 17: activity lifetime

CONFIRMED: The recovered lifetime body uses 520 bits.

The known safe state values are 3, 6, and 10.

The native state dispatcher uses an unbounded jump table.

Do not send an untested state value.

The body includes spawn overrides and a waiting-state switch.

The current neutral path uses state 3.

29.3 Type 18: activity script

The recovered type-18 schema identifier is `0x80809919`.

Type 18 shares a common state block with type 35.

This object is activity-script authority.

It is not the complete mission by itself.

29.4 Type 35: mission director

The recovered type-35 schema identifier is `0x808099BF`.

It uses the same common state block as type 18.

This object coordinates mission-director state.

It does not directly replace scene, actor, or objective objects.

29.5 Shared script state

The common shared-state schema is `0x808099C4`.

The recovered body contains these values:

- Active flag
- Five 64-bit values
- Final 32-bit value

The native upper time horizon observed in this body is `0x0000134F00C00000`.

This value represents approximately 365 days in the observed time unit.

An all-zero shared tail creates an empty validity window.

Use an explicit valid window in a neutral script body.

29.6 Type 8: configuration

The recovered neutral configuration body uses 35 bits.

29.7 Type 16: package state

The recovered neutral package body uses seven bits.

29.8 Type 41: queues

The recovered neutral queues body uses 12 bits.

29.9 Type 67: spawn keys

The recovered spawn-key body uses 1,057 bits.

29.10 Presentation support types

The current authored seed includes neutral forms for these broad presentation systems:

- Type 68: directive
- Type 53: dialogue
- Type 11: music

Neutral publication keeps the objects schema-valid without forcing a presentation edge.

30. Recovered mission object schemas

The following recovered neutral bodies come from current mission research.

They are examples of exact schema work.

They are not a general registry for all mission resources.

30.1 Recovered neutral definitions

- Type 1: schema 0x80807EC9, 24 neutral bits
- Type 4: schema 0x8080992F, 253 neutral bits
- Type 23: schema 0x80804F48, 147 neutral bits
- Type 26: schema 0x8080954B, 171 neutral bits
- Type 30: schema 0x80809532, 41 neutral bits
- Type 31: schema 0x80809524, 66 neutral bits
- Type 34: schema 0x8080956A, one neutral bit
- Type 43: schema 0x8080626B, 37 neutral bits
- Type 70: schema 0x808094F1, 23 neutral bits

The active form can be much longer than the neutral form.

For example, the observed active Omega type-1 spawner form used 355

bits.

30.2 Type 4 active field order

The recovered type-4 active body uses this field order:

1. 32-bit value
2. 32-bit value
3. Active Boolean
4. Resolve Boolean
5. 32-bit value
6. Nested key with 32-bit, 7-bit, and 16-bit parts
7. Three 32-bit vector values
8. Boolean
9. Inherited two-bit state
10. Optional 32-bit value

Do not reorder fields by their apparent semantic purpose.

Encode them in recovered wire order.

30.3 Type 43 selector

The recovered type-43 body contains these fields:

1. 32-bit selector
2. Active Boolean
3. Nested four-bit value

The inactive selector object must exist before an active edge.

Creating only the active body can miss native initialization work.

30.4 Type 1 spawner

The active type-1 example contains nested generation counts, request counts, and a squad-device reference.

A wrong network spawner can duplicate an actor that the native scene already creates.

Do not add a spawner only because an actor is absent from one frame.

First determine whether the scene owns that actor.

30.5 Unresolved active bodies

UNKNOWN: The active forms for the investigated type-26 and type-31 resources are not safely solved.

Do not activate them with guessed fields.

Keep the recovered neutral form until direct evidence identifies the active schema.

31. Mission seed and readiness

The authored mission seed publishes neutral schema-valid objects before active mission edges.

This process prepares the native managers.

31.1 Seed timing

Publish the seed after the world reaches the in-world phase.

Do not publish it during the initial world filter phase.

An early authored roster can remove or replace the first player citizen.

31.2 Seed setting

CURRENT SOURCE: `seedAuthoredSensors` controls authored neutral seeding.

The setting is off by default.

31.3 Readiness edge

Use host readiness as the first safe activation boundary.

Initialize the mission director, activity script, selector, and related neutral objects first.

Then change one mission latch or scalar.

Do not publish several guessed active states in one update.

31.4 Connection publication state

Keep publication stage and patch epoch on the connection.

If packet encoding or send fails, retain the previous stage.

Do not advance the stage before successful publication.

31.5 Deferred state mutation

Do not change host state inside an allocator or native object teardown callback.

The callback can hold a lock or iterate state that the mutation changes.

Queue the state change.

Apply it at the next safe server or connection update boundary.

32. Mission director and activity script

The mission director and activity script coordinate mission state.

They do not contain the complete mission graph by themselves.

32.1 Authority role

Type 35 provides mission-director authority in the recovered activities.

Type 18 provides activity-script authority.

These objects can publish mission latches, time windows, and shared state.

Other objects still own sensors, spawners, scenes, dialogue, objectives, and world devices.

32.2 Neutral initialization

Create each script object with a schema-valid neutral body.

Give its shared state a valid time window.

Confirm that the native runtime object exists.

Confirm that phase 2 applies the body.

Only then send an active edge.

32.3 Scalar progression

A recovered mission can use a scalar as a coarse stage value.

The number is resource-specific.

Do not create one global meaning for all missions.

For the investigated Omega opening, the observed Sunrise interpretation was:

- 0: inactive
- 1: ready

- 2: trigger accepted and scene active
- 3: completion after portal publication

This sequence is an Omega example. It is not a universal mission enum.

32.4 Change discipline

Change one script field per controlled test.

Keep the destination, roster, patch epoch, and scene selection constant.

Record the exact publication time and the first native response.

If several fields change together, the test cannot identify the required input.

32.5 Native lifecycle edges

Prefer an observed native lifecycle edge over a timer.

Useful edges include these events:

- Player enters a sensor
- Host becomes ready
- Scene reports a phase
- Actor factory retires a cast
- World device reports a state
- Objective manager accepts completion

A timer can reproduce one recording but fail on another machine or frame rate.

32.6 Mission handoff

Publish the next stable mission state before you mark the current script stage complete.

This order prevents a visible gap between the scene and the persistent

state.

The next stable state can be a world device, a final actor pose, an open portal, or an objective state.

33. Sensors and triggers

Sensors convert player or world activity into mission inputs.

The client sends observed sense data with activity-message type 6.

33.1 Use real trigger evidence

Use the client's type-6 sense update when it identifies a mission trigger.

Do not replace it with a fixed delay unless no trigger path is available.

The sense update proves that a native sensor evaluated the relevant condition.

33.2 Trigger identity

Identify a trigger with its complete object reference and group context.

Log these values:

- Registry group key
- Slot type
- Authored slot index
- Runtime handle and generation when available
- Bubble
- Patch epoch
- Sense bits
- Time relative to scene and mission state

33.3 Omega trigger example

CONFIRMED: The investigated Omega opening used client trigger D00142CF.

The observed object was type 30 at authored index 20.

This exact identity is mission-specific.

The general result is that a real type-6 edge can replace a guessed mission timer.

33.4 Debounce

A sensor can report the same condition many times.

Track the accepted edge in connection or activity state.

Do not restart a one-shot scene for every repeated sense packet.

Reset the latch only at a defined mission lifecycle boundary.

33.5 Trigger success boundary

A received sense packet proves that the client observed a condition.

It does not prove that the mission script accepted the condition.

Log the script state before and after trigger processing.

34. Scene system model

A scene is a layered runtime system.

It is not one animation object.

34.1 General scene layers

The observed scene path can contain these layers:

1. Scene authority object
2. Scene selector
3. Master timeline
4. Cast scheduler
5. Entity factory
6. Behavior and animation controller
7. Presentation object
8. Body, cloth, and head models
9. Authored event track
10. Transform bank
11. Transform provider
12. Effect objects
13. Renderer composite
14. Client phase report
15. Actor retirement

A scene can use only part of this list.

The ownership boundaries can also differ between scenes.

34.2 Scene authority

The server usually activates a scene through a mission object or selector.

The native client then constructs its timeline and cast.

Do not start with a direct actor spawn when a native scene owns the actor.

A separate spawner can duplicate the native scene cast.

34.3 Selector initialization

Create the selector in its inactive form before the active edge.

This step lets the native client initialize its dependent scene objects.

Then change the selector or active flag in one update.

34.4 Scene phase report

The client can report a scene handoff through an object sense body.

For the investigated Omega scene, the type-43 body changed between 108 and 140 bits.

The 140-bit state armed the handoff.

It did not by itself prove final completion.

The native actor retirement occurred later and supplied the observed completion edge.

34.5 Dialogue is a separate proof

Dialogue audio can play without mission actors.

This result proves that the dialogue or audio path ran.

It does not prove that the scene factory created the cast.

It does not prove that the animation timeline ran.

34.6 Scene isolation test

An A/B cast isolation test is a high-value scene test.

Use this procedure:

1. Enable no scene casts.
2. Record audio, actors, effects, and mission state.
3. Enable one cast or scene entry.
4. Repeat the same route.
5. Compare the first construction and retirement events.
6. Add one additional scene entry only after the first result is stable.

This test identifies which scene owns each visible or lifecycle result.

34.7 Scene numbers are local labels

Recorder labels such as `Scene 1`, `Scene 2`, and `Scene 3` describe one investigation.

They are not engine-wide scene types.

Factory call numbers are also recorder-local.

Do not convert them into actor identities.

35. Scene phase roles

Several scene casts can use the same character, entity root, or presentation definition.

They can still have different mission roles.

35.1 Entry scene

An entry scene establishes the actor and begins the action.

It can create the visible character and start the first animation.

35.2 Transition scene

A transition scene can own authored events and temporary effects.

It can also create a cast body that exists only during the transition.

35.3 Persistent scene

A persistent scene or state keeps the actor in its final mission pose.

It can outlive the transition effects.

35.4 Omega phase example

CONFIRMED: The isolated Omega opening produced these case results:

- Scene 1 produced the correct Ikora action without the purple effect defect.
- Scene 2 produced the purple aura and beams with an unwanted cast body.
- Scene 3 produced the stable final character state.
- No enabled cast produced dialogue without mission actors.

This result proved that the scene roles were separate.

It did not prove that one scene object owned every effect transform.

35.5 Retained transition scene

A transition scene can be retained after a later persistent scene starts.

This method can keep a temporary effect visible.

Before you retain it, identify the normal retirement side effects.

If retirement normally completes the mission stage, a permanent scene suppresses that completion edge.

Add one controlled substitute at the same semantic boundary.

Do not mark the mission complete immediately when you start the retained scene.

36. Actor and entity construction

A scene actor can pass through several factories before it becomes

visible.

The actor authority, animated entity, and render models can be separate objects.

36.1 Actor identity

Record these values for an actor:

- Scene handle and generation
- Cast entry
- Entity handle and generation
- Definition tag
- Presentation handle
- Root transform
- Model resources
- Construction time
- Retirement time

Do not identify an actor by a pointer suffix or low handle index.

The allocator can reuse both values.

36.2 Root transform

The entity root transform gives the broad object position and orientation.

It is not a bone or socket pose.

An effect attached to a hand, weapon, head, or marker can use a different transform provider.

Changing the root can move the actor without moving a socket-driven effect.

36.3 Native scene actor

A native scene can construct its own actor from the cast entry.

Do not add a network spawner until a capture proves that the scene has no cast factory for that actor.

36.4 Network spawner

A type-1 authority object can request a squad or entity spawn.

This path can be correct for a world actor that is not scene-owned.

It can be incorrect for an actor that the timeline already owns.

A duplicate actor can appear in a default pose or at an unrelated root.

36.5 Actor retirement

Retirement can be a mission signal.

Log it as a lifecycle event, not only as object cleanup.

Do not suppress retirement without checking mission state after the normal retirement frame.

37. Model and presentation construction

The visible character can contain several model resources.

The body, head, cloth, and attachments can have separate construction calls.

37.1 Presentation ownership

The scene can own one presentation object that creates several render models.

Suppressing one body model does not necessarily suppress the head.

Use the exact nested resource or construction call for each model piece.

37.2 Exact suppression

When a scene has wanted events but an unwanted cast body, suppress only the unwanted model construction.

Keep the scene authority, timeline, authored events, and effects active.

Filter with exact stable evidence such as the complete resource tag and scene context.

Do not suppress every model of the same character globally.

Another scene can use the same character correctly.

37.3 Omega model example

CONFIRMED: Suppressing the identified Scene 2 body kept the purple authored effects.

The head remained because it used a separate model construction path.

Suppressing the second exact resource removed the head.

This test proved that the unwanted body and the wanted effects had separate construction owners.

37.4 Render suppression boundary

Model suppression removes presentation.

It does not remove the scene actor, its animation state, or its transform providers.

This separation can be useful when the actor must continue to drive authored events.

It can also leave invisible collision or logic if those systems belong to the actor.

Test presentation and gameplay separately.

38. Visual effects

Visual effects can be authored by a scene event, a mission object, or a persistent world device.

Do not assume that the visible character owns the effect.

38.1 Effect ownership questions

Answer these questions in order:

1. Which scene or mission object creates the effect?
2. Which event creates each effect handle?
3. Which definition does the handle use?
4. Which transform bank does the event select?
5. Which provider resolves the selected transform?
6. Which composite object writes the renderer input?
7. Which lifecycle event destroys or disables the effect?

38.2 Complete effect identity

Log these values for every candidate effect:

- Full effect handle
- Handle generation
- Effect definition
- Parent scene handle
- Authored event index
- Transform-bank selector
- Provider address or handle
- Parent composite

- Creation time
- First render time
- Retirement time

Do not group effects only by color or approximate position.

38.3 Transform bank

A transform-bank selector is an index into an authored transform source.

It is not an actor number.

A value such as one, two, or three does not mean Scene 1, Scene 2, or Scene 3.

The bank can contain roots, markers, sockets, and authored fixed transforms.

38.4 Provider selection

The provider can select a socket or marker before the event callback runs.

Changing the callback actor after provider selection can have no effect.

Capture the provider at the bank-resolution boundary.

38.5 Direct-bank route

Some effects use a direct-bank route.

That route can ignore the callback object that appears to own the event.

A late actor substitution cannot change a provider that the direct-bank route already selected.

38.6 Composite renderer state

An effect object can contain several transform copies.

A later composite can calculate the final renderer input from another source.

A successful memory write and readback prove only that one field changed.

They do not prove that the renderer used that field.

Before a coordinate experiment, identify these items:

- Parent composite
- Final endpoint buffer
- Later rewrite functions
- Actual renderer input

38.7 Separate effect consumers

An aura and several beams can use different effect handles and providers.

One transform change can move a beam endpoint but not the aura.

Filter each definition separately.

Then compare their shared parent or provider.

38.8 Scene effect and mission visual

A scene-authored effect and a post-scene mission visual can look similar.

They can be different systems.

For example, a transition beam can come from a scene event while the final portal comes from a type-4 mission object.

Do not use appearance alone to assign ownership.

38.9 Bounded transform experiment

Use this procedure for a transform hypothesis:

1. Filter an exact small set of effect handles.
2. Capture the original provider and final renderer transform.
3. Apply one large visible offset after the suspected composer.
4. Confirm that the write applied.
5. Check for a later rewrite.
6. Check the visual result once.
7. Stop the route when the final renderer input does not change.

Do not repeat offsets at lower object layers after the renderer boundary disproves them.

38.10 Omega VFX result

DISPROVEN: A direct transform-bank context substitution attached the purple effects to the visible Ikora pose.

DISPROVEN: Rewriting the investigated Scene-bank position by plus 10 moved the rendered effect.

DISPROVEN: Rewriting the filtered live effect position vectors after the suspected composer moved the rendered result.

The effect remained detached from the visible Ikora character.

The tests narrowed the unresolved owner to a later or separate composite path.

They did not show that the visible Ikora model itself owns the effect.

UNKNOWN: The exact final socket or composite binding for this case remains unresolved.

39. World devices, portals, and barriers

A visual portal, a collision barrier, and a transition animation can be separate objects.

Treat them as separate state owners.

39.1 Closed and open states

A closed wall can exist before a transition scene.

The transition scene can remove or change the wall.

The open portal can then exist as a persistent state.

Do not assume that the transition scene creates the closed wall.

39.2 Registry ownership

A world device usually needs a registry that the active scenario authority owns.

Finding the device definition in another package does not mount its registry.

Publishing its type and index in phase 1 also does not guarantee construction.

39.3 Foreign-registry test

CONFIRMED: An investigated wall test published the intended foreign registry identity in phase 1.

The client runtime-object count did not increase.

No matching runtime object appeared.

This result placed the failure before the device body and renderer.

The active mission authority container did not construct the foreign

object.

39.4 Stop rule

Stop changing the auth body when the runtime factory creates no object.

The body cannot activate an object that does not exist.

39.5 Remaining construction routes

The remaining general routes are:

1. Mount the destination registry through the native registry lifecycle.
2. Instantiate the exact native entity through its native factory.
3. Rebind an existing mission-owned slot when its schema and lifetime are compatible.

Each route needs separate proof.

Do not describe a registry read as a registry mount.

39.6 Collision and presentation

The visual wall and its collision can use different objects.

A renderer-only wall can look correct but permit the player to pass.

A collision-only wall can block the player while remaining invisible.

Test both outputs.

40. Mission completion and progression

Mission progression depends on semantic lifecycle edges.

Do not infer completion only from a visible final frame.

40.1 Completion sources

A mission stage can complete from one or more of these sources:

- Script scalar or latch
- Sensor result
- Scene phase report
- Actor retirement
- Spawner population result
- World-device state
- Objective completion
- Activity-host manager state

40.2 Presentation before completion

Publish the next persistent presentation before the current transition retires.

Then publish or accept completion.

This order keeps the scene and world state continuous.

40.3 Suppressed retirement

If a diagnostic hook keeps a scene alive, it can suppress actor retirement.

The mission can then wait forever for the missing retirement edge.

Add a deferred completion only after you prove that retirement supplied the original edge.

Keep the substitute at the same semantic boundary.

40.4 Infinite scene rule

An infinite scene is a presentation policy.

It must not become an infinite mission wait.

Track these states separately:

- Scene presentation retained
- Scene logical handoff accepted
- Mission stage completed
- Persistent successor state active

40.5 Progression proof

Use the owning mission state as the final proof.

A scene that looks complete is presentation evidence.

A changed mission director or objective state is progression evidence.

41. AI, encounters, and objectives

Mission reconstruction is not complete when the opening scene works.

AI and objectives require additional authority systems.

41.1 Actor animation is not AI

A scene actor follows a timeline or behavior graph.

An AI combatant needs authority, goals, pathing, targeting, and population lifecycle.

Do not describe an animated scene actor as working AI.

41.2 Spawner is not encounter control

A spawner can request entities.

An encounter also needs wave conditions, population accounting, retirement, objective gates, and failure handling.

41.3 Objective presentation is not objective logic

A directive or HUD message can appear without a functional objective.

Test the objective owner, completion condition, and next-stage transition.

41.4 Current general gap

UNKNOWN: Sunrise does not have a complete general mission-graph interpreter.

UNKNOWN: The complete AI authority and encounter system is not reconstructed.

UNKNOWN: General objective, reward, and quest-completion publication is incomplete.

Treat each recovered mission as a bounded authored state machine until these systems exist.

42. State and transaction design

Mission reconstruction creates many related state objects.

A partial mutation can leave the client and server in different generations.

Use explicit ownership and transactions.

42.1 Process-owned state

Store these items in process-owned `State`:

- Activity sessions
- Destinations
- Entity-slot leases
- Membership records
- Bubble grants
- Matchmaking advertisements
- Group-host mappings
- Build-data catalogs
- Durable mission stage for the active session

Do not store a pointer into a request buffer.

42.2 Connection-owned state

Store these items on the connection:

- Patch epoch
- Publication stage
- Transition token observation
- Warmup count
- Last published roster generation
- Trigger debounce for connection-scoped input
- Pending notification schedule

Clear this state when the owning connection ends.

42.3 Client-native state

Keep native handles, pointers, and executable RVAs in the client integration layer.

Do not put a native pointer in cross-connection server state.

The native object can retire or the allocator can reuse its address.

42.4 Prepare and commit

Use a prepare-and-commit operation for each mutation that can fail after

validation.

The prepare phase reads current state and creates a value-only plan.

The commit phase validates revisions and applies the plan under the correct lock.

42.5 Monotonic revisions

Use monotonic nonzero revisions.

Do not wrap a revision.

An exhausted revision allocator is safer than a stale transaction that becomes valid again.

42.6 Notification after commit

Create outbound notifications from committed state.

Do not publish a body that describes a mutation which can still fail.

If a send fails, use the prior committed state and a retry marker.

42.7 Deferred native callbacks

A native hook can run inside an allocator, renderer, or object destructor.

Do not acquire broad server locks or rebuild state inside that callback.

Copy the minimum stable evidence to a fixed record.

Process the record at a safe update point.

42.8 Capacity behavior

Every bounded table needs an explicit full-table rule.

The rule can reject, evict, or reuse a compatible record.

Log the selected rule and victim identifier.

Silent eviction can appear as a random later mission failure.

43. Observability model

Logs must answer ownership, order, and postcondition questions.

High-volume pointer logs do not answer these questions by themselves.

43.1 Correlation key

Give each activity run a correlation key.

Include it in destination, session, group, roster, mission, scene, and effect records.

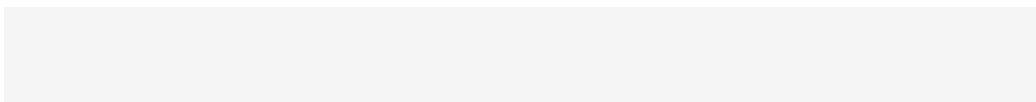
At minimum, use these values:

- Process start identifier
- Activity-session identifier
- Activity-host identifier
- Group-session identifier
- Connection identifier
- Transition token
- Roster generation
- Scene handle and generation

43.2 Structured event form

Use a stable event name and key-value fields.

Example:



```
mission.scene.actor_constructed activity=12 scene=0x... g
```

Keep the same field name for the same concept across all subsystems.

Do not alternate between `sid`, `session`, and `activity_id` for one identity.

43.3 State transition log

For every important transition, record these values:

- Owner
- Previous state
- New state
- Input event
- Object identity
- Revision or generation
- Result
- Time

43.4 Postcondition log

After a native call, read the owning state and log the postcondition.

Do not log only `call returned true`.

Useful postconditions include these values:

- Manager active flag
- Runtime-object count
- Current selector
- Actor count
- Model count
- Effect provider
- Renderer transform
- Mission stage

43.5 Absence log

An expected object that does not appear is important evidence.

Emit one bounded summary after the observation window.

Example:

```
mission.roster.runtime_absent group=0x... type=4 index=12
```

Do not emit the same absence every frame.

43.6 Rate control

Use counters and first-seen or state-change logs for high-frequency callbacks.

Sample stable per-frame data.

Always log a generation change, owner change, or lifecycle boundary.

43.7 Exact effect filter

An effect log should filter by exact definition and full handle.

Color, approximate position, and pointer suffix are weak filters.

43.8 Evidence archive

Archive these items for each important A/B test:

- Source diff
- Source revision
- Built DLL hash
- Installed DLL hash
- Settings
- Sunrise log

- Old Sunrise log when relevant
- Screenshot or video
- Short expected-result statement
- Actual-result statement
- Evidence label

This archive makes a negative result reusable.

44. Runtime recorder design

A recorder must capture one ownership boundary.

Do not add a recorder to every nearby function.

44.1 Recorder question

Write one sentence before implementation.

Example:

Record which Scene 1 runtime entry constructs pose object `BC30` and which entry constructs `AAD0`.

If the question cannot fit in one sentence, split the recorder.

44.2 Recorder fields

A useful recorder contains these fields:

- Event kind
- Full runtime handle
- Generation
- Definition tag
- Parent handle
- Scene handle
- Actor or cast context
- Input object

- Output object
- Relevant transform
- Thread identifier
- Relative time

Use fixed-size records in a hot hook.

Format them outside the callback when possible.

44.3 Entry and exit records

Record both input and output when a function constructs or resolves an object.

An entry record alone cannot prove which object the function returned.

An exit record alone can lose the input owner.

44.4 Provenance chain

Build a chain from the first stable owner to the final consumer.

For an effect transform, the chain can be:

```
scene event -> bank selector -> provider -> effect instan
```

Stop when the chain reaches the actual consumer.

44.5 Minimal duration

Record from before the target scene starts until after its normal retirement.

Do not capture a complete mission when the target occurs in a ten-second window.

Smaller captures reduce handle reuse and unrelated noise.

44.6 Build identity

Log the executable image identity and expected RVA set at recorder startup.

Disable the recorder when the identity does not match.

Do not call a pinned RVA on an unverified executable.

45. Static reverse engineering

Static analysis supplies schemas, ownership candidates, and call relationships.

Runtime analysis supplies identities, order, and postconditions.

Use both methods together.

45.1 Analysis image

The current unpacked analysis image is:

```
C:\Destiny 2 Development\destiny2_unpacked.bin
```

The observed analysis base is:

```
0x7FF618070000
```

For the current image, file offset equals RVA.

This equality is an observed property of this image.

Revalidate it for another executable.

45.2 RVA rule

An RVA is build-specific.

Before use, verify these items:

1. Executable hash
2. Module base
3. Function bytes or signature
4. Calling convention
5. Expected references or native strings

45.3 Schema recovery

Recover a bit schema from both the reader and writer when possible.

Record these details:

- Field order
- Field width
- Signedness
- Bias
- Presence condition
- Nested length
- Default value
- Bounds check
- State side effect

45.4 Native strings

Native strings can identify managers, parameter names, failure reasons, and state labels.

Use them as navigation evidence.

Do not assign full semantics from a string alone.

45.5 Jump tables

An unbounded jump table makes unknown enum values dangerous.

Use only values that a valid capture or controlled trace confirms.

45.6 Protected code

VMProtect or another protector can hide a decoder or state machine.

When static recovery stops, capture its inputs and outputs at a stable unprotected boundary.

Do not guess a large protected payload from its header alone.

46. Build and deployment procedure

Use one repeatable build and deployment path.

The current development helper is:

```
powershell -ExecutionPolicy Bypass -File "C:\Destiny 2 De
```

46.1 Before the build

1. Close the game before DLL replacement.
2. Record the source revision.
3. Record the working-tree diff.
4. Record untracked probe files.
5. Set the intended runtime settings.

A normal `git diff` does not include untracked files.

List them separately.

46.2 After the build

1. Confirm that compilation succeeded.
2. Record the built DLL hash.
3. Record the installed DLL hash.
4. Confirm that both hashes agree.

5. Start the game.
6. Confirm the expected startup marker in the log.

46.3 Logs

The primary logs are:

- `C:\Destiny 2
Development\bin\x64\Sunrise\logs\sunrise.log`
- `C:\Destiny 2
Development\bin\x64\Sunrise\logs\sunrise.log.old`

The current log contains the latest process.

The old log can contain the completed prior run after a restart.

Confirm process-start time before you attribute an event to a test.

46.4 Deployment proof

Do not use a screenshot alone to prove that a new hook ran.

Add one unique startup or activation marker.

Verify the marker and installed DLL hash.

Then interpret the visual result.

47. Bounded experiment method

A bounded experiment tests one causal statement.

It has a defined stop condition.

47.1 Required statement

Write these items before the test:

- Observation
- Hypothesis
- Single changed input
- Expected positive result
- Expected negative result
- Stop condition
- Rollback action

47.2 Baseline

Run an unchanged baseline after every large code or settings change.

The baseline confirms that the route and capture still work.

47.3 One variable

Change one owner, field, resource, or lifecycle edge.

Do not combine a scene filter, transform rewrite, and mission-stage change in one test.

47.4 Large visible probe

For a visual transform, use a large temporary offset.

A small offset can be hidden by camera angle or effect width.

Use the large offset once.

Remove it after the result.

47.5 Binary result

Prefer a test with a binary result.

Example:

- Runtime factory call occurs
- Runtime factory call does not occur

A binary result removes broad interpretation.

47.6 Negative result

A negative result must remove a branch from the investigation.

Write the branch as `DISPROVEN`.

Do not rephrase it as a reason to repeat a nearby form of the same test.

47.7 Stop condition

Stop a route when the test reaches its consumer and the predicted output does not change.

Move to another owner or architecture route.

47.8 Cleanup

Remove obsolete probes after their result is archived.

Old probes can change timing, fill logs, or hide the active test.

Keep only the probes that support current operation or a documented diagnostic mode.

48. Layer-specific diagnostic guide

Use this guide to select the next observation.

48.1 No secure service activity

Check platform bootstrap, BAP framing, server hello, and pending responses.

48.2 Service 6 succeeds but no world loads

Check service 7 identifier, destination record, package mapping, and loader state.

48.3 World loads at the wrong location

Check arrival bubble, initial slice, spawn hash, package availability, and membership region.

48.4 World loads but screen remains black

Check world phase, native spawn gate, spawn hold, and fade-channel arm time.

48.5 Public search does not start

Check the private or public decision at the region transition boundary.

Then check configuration presence, advertisement readiness, and ambassador identity.

48.6 Peer connects but group does not join

Check application-ready, membership revision, self `NetAddr`, join identifier, and parameter publication.

48.7 Group joins but no activity host appears

Check the nonzero `activityHost` parameter, service 16 response, regional host mapping, and advertisement host identifier.

48.8 Activity host joins but no roster appears

Check type-18 refresh handling, type-1 state, type-12 membership, type-54 grants, and type-5 phase 1.

48.9 Phase 1 names a group but no runtime object appears

Check scenario ownership, registry mounting, slot descriptor, bubble context, and native factory calls.

Do not edit the object body yet.

48.10 Runtime object exists but body does not apply

Check patch epoch, state sequence, object-reference bias, slot flags, and exact bit remainder.

48.11 Mission object applies but scene does not start

Check neutral selector initialization, active edge, mission scalar, sensor input, and native scene-authority object.

48.12 Dialogue plays but no actor appears

Check cast scheduler, entity factory, presentation construction, and model filters.

Do not change the audio route.

48.13 Actor appears in a default pose

Check whether a separate network spawner duplicated a native scene actor.

Then check animation-controller binding and scene cast ownership.

48.14 Effect appears at the wrong location

Check effect definition, authored event, transform selector, provider, composite, later rewrite, and renderer input.

Do not infer the provider from the actor root.

48.15 Visual device is absent

Check registry mount and runtime-object construction before the body and renderer.

48.16 Scene looks correct but mission does not progress

Check phase report, actor retirement, mission scalar, world-device state, and objective completion.

49. Disproved and unsafe investigation patterns

This section records patterns that caused false progress or repeated loops.

Do not repeat them without new evidence.

49.1 Package equals runtime

DISPROVEN PATTERN: A package read proves that its object can be published in the active mission roster.

A package definition can exist outside the mounted mission registry.

49.2 Phase 1 equals construction

DISPROVEN PATTERN: A type-5 phase-1 record creates the named object.

Phase 1 can decode and name the group while the runtime factory creates nothing.

49.3 Accepted packet equals applied state

DISPROVEN PATTERN: A valid service or group response proves that the native manager became active.

Always inspect the owning manager postcondition.

49.4 Function success equals semantic success

DISPROVEN PATTERN: A native function return value of true proves the requested lifecycle state.

The Homecoming manager activation returned true while the active flag stayed zero.

49.5 Audio equals scene

DISPROVEN PATTERN: Dialogue playback proves that mission actors and animations exist.

Audio can run through a separate presentation path.

49.6 Character model equals effect owner

DISPROVEN PATTERN: An effect near a character must use the

character model transform.

The effect can use a scene bank, marker provider, fixed transform, or separate composite.

49.7 Root equals socket

DISPROVEN PATTERN: The actor root transform is the hand or attachment transform.

A socket-driven effect uses the animated pose or another provider.

49.8 Memory write equals renderer change

DISPROVEN PATTERN: A successful write and readback prove a visual change.

The renderer can use another copy or rewrite the value later.

49.9 Low handle index equals stable object

DISPROVEN PATTERN: A low handle index or pointer suffix identifies an actor or effect across frames.

Handle and pointer reuse make this filter unsafe.

49.10 Recorder factory number equals actor number

DISPROVEN PATTERN: Recorder-local factory order identifies `Actor 1`, `Actor 2`, or one semantic cast.

Use definitions, parents, generations, and time.

49.11 Final state write equals lifecycle

UNSAFE PATTERN: Force the final member, manager, scene, or

mission state directly.

The client can need side effects from intermediate edges.

49.12 Zero means neutral

UNSAFE PATTERN: Fill an unknown body with zero.

Biased references, required selectors, and time windows can make zero invalid.

49.13 Present empty means omitted

UNSAFE PATTERN: Send an empty optional protobuf message as a neutral placeholder.

Presence can activate policy with zero values.

49.14 Timer equals trigger

UNSAFE PATTERN: Replace an available native sensor or retirement edge with a guessed timer.

Timing changes across machines and repeated runs.

49.15 Keep scene alive without completion plan

UNSAFE PATTERN: Prevent scene retirement and expect normal mission completion.

Retirement can be the completion signal.

49.16 Change several layers in one build

UNSAFE PATTERN: Change transport, roster, scene, and VFX code in one test.

The result cannot identify which change caused the behavior.

49.17 Continue after the consumer disproves the route

UNSAFE PATTERN: Apply the same transform idea at more upstream objects after the final renderer input does not change.

The route has reached its stopping point.

Select another owner.

50. General mission reconstruction playbook

Use this playbook for a new activity.

Do not start with mission VFX or actor hooks.

50.1 Phase A: establish the baseline

1. Record the executable and DLL hashes.
2. Load the activity without mission overrides.
3. Record the destination request and response.
4. Record the world, arrival, and public or private route.
5. Save one complete baseline log and screenshot.

Exit condition:

The same baseline result occurs in two runs.

50.2 Phase B: establish transport and sessions

1. Confirm the activity-session identifier.
2. Confirm the gameplay endpoint.
3. Confirm peer application-ready.
4. Confirm group membership and self-member resolution.

5. Confirm the nonzero activity-host parameter.
6. Confirm service 16 and service 17.
7. Confirm activity membership.

Exit condition:

The client has stable group and activity-host membership.

50.3 Phase C: establish world state

1. Confirm package and scenario selection.
2. Confirm arrival bubble.
3. Confirm initial slice.
4. Confirm spawn-set package availability.
5. Confirm world `Arrived` state.
6. Confirm the player spawn and fade release.

Exit condition:

The player enters the correct world at a reproducible arrival.

50.4 Phase D: establish the core roster

1. Publish global activity state.
2. Publish membership.
3. Publish bubble authority.
4. Publish safe top-level roster groups.
5. Publish the current bubble-local groups.
6. Confirm native runtime-object construction.
7. Publish phase-2 bodies with the exact patch epoch.

Exit condition:

Participation, lifetime, and script objects exist and accept their neutral bodies.

50.5 Phase E: establish mission control

1. Initialize type 18 and type 35 with valid shared state.
2. Initialize required sensors and selectors.
3. Wait for host readiness.
4. Accept one real client sense edge.
5. Change one mission scalar or latch.
6. Confirm the owning manager postcondition.

Exit condition:

One native mission transition occurs from a real input.

50.6 Phase F: isolate presentation

1. Run with no candidate scene cast.
2. Add one scene entry.
3. Record audio, actors, models, effects, and retirement.
4. Add one additional entry.
5. Assign each output to an owner.

Exit condition:

Each required presentation output has a known scene or mission owner.

50.7 Phase G: establish completion

1. Record the native phase report.
2. Record actor retirement.
3. Record mission scalar changes.
4. Record world-device and objective changes.
5. Identify the first accepted completion edge.
6. Confirm the next stable state before completion.

Exit condition:

The mission advances without a diagnostic timer.

50.8 Phase H: add world devices and encounters

1. Confirm registry ownership.
2. Confirm runtime-object construction.
3. Apply a neutral body.
4. Activate one state.
5. Test visual and collision output separately.
6. Add AI or objective control only after its authority owner is known.

Exit condition:

The device or encounter has presentation, gameplay, and lifecycle proof.

50.9 Phase I: remove diagnostic debt

1. Remove large coordinate probes.
2. Remove unused hooks.
3. Disable high-volume recorders.
4. Convert required exceptions to named settings.
5. Document every remaining build-specific RVA.
6. Run the complete route twice.

Exit condition:

The mission route works without hidden probe dependencies.

51. General lessons from Homecoming

Homecoming exposed the difference between network lifecycle and mission simulation.

51.1 Corrected handshake assumption

HISTORICAL: The phrase `ready for instantiation` also occurred in a normal working farm route.

Therefore, that phrase was not a Homecoming failure signal.

The empty activity-host session identifier and roster state 3 were also not sufficient diagnostics by themselves.

General lesson:

Compare a suspect route with a known working route before you assign meaning to a log phrase.

51.2 Public transition decision

HISTORICAL: Changing the exact region transition input from private to public started the citizen path.

The change also started matchmaking search behavior.

General lesson:

Patch the decision input, not a global reader or a later result.

51.3 Ambassador identity

HISTORICAL: An ambassador identity equal to the local identity selected a local ambassador or search path.

A different valid ambassador identity selected the citizen peer route.

General lesson:

Identity relationships can select protocol branches even when each identifier is individually valid.

51.4 Matchmaking configuration

HISTORICAL: A valid configuration materialized the expected matchmaking lane.

An empty present configuration caused immediate policy behavior.

General lesson:

Optional message presence is an input. It is not formatting only.

51.5 Manager route

HISTORICAL: The authored manager progressed through modes 1, 2, 4, and 5.

Observed payload sizes included 0x17F8, 0x1880, 0x90, and 0xA0.

The manager committed route 1 with the same nonce.

These events proved that the manager route existed.

They did not prove that the mission simulation became active.

51.6 Kind-22 activation

HISTORICAL: A kind-22 activation reached the manager and returned true.

The manager active flag stayed zero because required identity blocks were empty.

General lesson:

Transport success, decode success, route success, and manager activation are four separate proofs.

51.7 Run-local labels

The recorder name `identity2` described one run-local manager label.

It was not a stable protocol identity name.

General lesson:

Keep recorder labels separate from recovered engine terms.

51.8 Activity metadata

The minimal 128 zero bytes in service 7 did not prevent the observed manager from reaching route 1.

Therefore, that metadata block was not the immediate route gate in that test.

This result does not prove that the block is unused in every activity.

52. General lessons from Omega

Omega exposed the separation between mission authority, scene phases, presentation models, effects, and world devices.

52.1 Mission script

CONFIRMED: Neutral authored mission objects could exist before the active opening edge.

CONFIRMED: A real client sense update could supply the opening trigger.

General lesson:

Seed valid neutral objects before you publish one active edge.

52.2 Scene isolation

CONFIRMED: Three isolated scene entries had distinct entry, transition, and persistent roles.

General lesson:

Shared character content does not imply shared scene ownership.

52.3 Model suppression

CONFIRMED: Exact body and head construction filters removed unwanted presentation while the scene effects remained.

General lesson:

Suppress the narrow presentation resource when the timeline and authored events are still required.

52.4 Effect binding

DISPROVEN: The investigated actor and pose substitutions attached the purple effect to the visible character.

DISPROVEN: The investigated direct coordinate writes moved the final rendered result.

General lesson:

Trace an effect through its final renderer composite before more coordinate experiments.

52.5 Scene retention

CONFIRMED: A transition scene could remain present while the persistent scene also ran.

This retained presentation can suppress normal retirement.

General lesson:

Separate retained presentation from logical scene completion.

52.6 Wall construction

CONFIRMED: A foreign registry record could appear in phase 1 without a new runtime object.

General lesson:

Registry mounting and runtime construction are earlier requirements than an auth body.

53. Current implementation boundaries

This section separates current source from historical results and open work.

53.1 Current source strengths

CURRENT SOURCE includes these substantial systems:

- BAP framing and secure service routing
- Transactional activity-session allocation
- Destination parsing and descriptor preservation
- Forced destinations
- Activity-host endpoint resolution
- Matchmaking request parsing and core response shapes
- Gameplay endpoint and peer transport
- Group membership and parameters
- Activity-message routing
- Entity-slot transactions
- Activity membership
- Bubble-authority grants
- Scenario and spawn-set extraction
- Roster phase-1 and phase-2 encoding
- Several recovered mission object bodies
- Spawn hold and selected native client hooks

53.2 Current source cautions

The worktree contains local changes and untracked files.

The named Git revision does not include those files.

The current session-search response is empty.

Historical descriptor search behavior can differ from this source.

The exact Omega authored-key rule is case-specific.

Several settings that enable authored mission behavior are off by default.

53.3 Known gaps

UNKNOWN or incomplete areas include these systems:

- Complete general mission-graph interpretation
- General AI authority and pathing
- General encounter and wave control
- Complete objective and reward progression
- General quest completion
- Complete schema coverage for every roster slot type
- Active schemas for several recovered mission objects
- Foreign registry mounting into an active mission authority container
- General native entity instantiation lifecycle
- Exact final socket binding for the investigated Omega purple effects
- Complete Demonware QoS payload structure
- Stable manager identity construction for the historical Homecoming activation route
- Build-independent native hook locations

53.4 Architectural ceiling

Some missing features cannot be solved by another packet field.

For example, a foreign device cannot accept an auth body when the native authority container never constructs it.

At that boundary, choose one of these actions:

1. Recover the native registry-mount lifecycle.
2. Recover the native entity factory lifecycle.
3. Reuse a compatible mounted object with full schema proof.
4. Accept a temporary client-executor bridge.

State the selected architecture before implementation.

54. Source map

Use this section to find the current implementation.

All paths are relative to `C:\Destiny 2 Development\Sunrise-src`.

54.1 BAP frames and routing

- `Sunrise/src/middleware/bap/frame.h`
- `Sunrise/src/server/bap/encrypted/routing/bap_service_routing.cpp`
- `Sunrise/src/server/bap/encrypted/transactions/service_outcome_commit.cpp`

These files define service identifiers, route selection, and staged commits.

54.2 Destination and activity allocation

- `Sunrise/src/server/bap/encrypted/activity_host_manager/activity_host_manager_route.cpp`
- `Sunrise/src/middleware/bap/activity_host_manager/request/activity_manager_request.cpp`
- `Sunrise/src/middleware/bap/activity_host_manager/request/selection/activity_manager_selection_parser.cpp`
- `Sunrise/src/middleware/bap/activity_host_manager/`

- request/selection/
activity_manager_selection_descriptor.cpp
- Sunrise/src/middleware/bap/activity_host_manager/
response/activity_manager_response.cpp
- Sunrise/src/state/activity/definition.h
- Sunrise/src/state/activity/destination/
definition.h
- Sunrise/src/state/activity/destination/
activity_destination_validation.cpp
- Sunrise/src/state/activity/destination/
activity_destination_spawn_binding.cpp
- Sunrise/src/state/activity/forced/
activity_forced_destination.cpp

54.3 Matchmaking

- Sunrise/src/middleware/bap/matchmaking/
definition.h
- Sunrise/src/middleware/bap/matchmaking/request/
matchmaking_request_parser.cpp
- Sunrise/src/middleware/bap/matchmaking/response/
matchmaking_response_encoder.cpp
- Sunrise/src/server/bap/encrypted/matchmaking/
matchmaking_route.cpp
- Sunrise/src/state/matchmaking/definition.h
- Sunrise/src/state/matchmaking/transactions/
matchmaking_prepare.cpp
- Sunrise/src/state/matchmaking/transactions/
matchmaking_commit.cpp

54.4 Gameplay advertisement and group host

- Sunrise/src/server/gameplay/
gameplay_advertisement.cpp
- Sunrise/src/server/gameplay/group/group_host.cpp
- Sunrise/src/server/gameplay/group/
group_host_sessions.cpp
- Sunrise/src/middleware/gameplay/group/

parameter_registry.h

54.5 Activity-message routing

- Sunrise/src/server/bap/encrypted/activity_message/activity_message_route.cpp
- Sunrise/src/middleware/bap/activity_message/activity_message_request_parser.cpp
- Sunrise/src/middleware/bap/activity_message/activity_message_notification_encoder.cpp
- Sunrise/src/server/bap/encrypted/push/activity/activity_message_push.cpp

54.6 Global state and membership

- Sunrise/src/middleware/bap/activity_message/activity_global_state_encoder.cpp
- Sunrise/src/middleware/bap/activity_message/activity_replicate_membership_encoder.cpp
- Sunrise/src/middleware/bap/activity_message/activity_membership_member_writer.cpp
- Sunrise/src/middleware/bap/activity_message/activity_membership_region_writer.cpp
- Sunrise/src/server/bap/encrypted/push/activity/activity_membership_push.cpp
- Sunrise/src/state/activity/membership/definition.h
- Sunrise/src/state/activity/membership/transactions/activity_membership_prepare.cpp
- Sunrise/src/state/activity/membership/transactions/activity_membership_commit.cpp

54.7 Entity slots and bubble authority

- Sunrise/src/middleware/bap/activity_message/activity_entity_slots_encoder.cpp
- Sunrise/src/middleware/bap/activity_message/

- `activity_entity_slots_decoder.cpp`
- `Sunrise/src/state/activity/entity_slots/definition.h`
- `Sunrise/src/state/activity/entity_slots/transactions/activity_entity_slot_prepare.cpp`
- `Sunrise/src/state/activity/entity_slots/transactions/activity_entity_slot_commit.cpp`
- `Sunrise/src/state/activity/bubble_authority/definition.h`
- `Sunrise/src/state/activity/bubble_authority/transactions/activity_bubble_authority_grant.cpp`

54.8 Scenario and roster extraction

- `Sunrise/src/client/content/scenarios/scenario_build.cpp`
- `Sunrise/src/client/content/scenarios/scenario_collect.cpp`
- `Sunrise/src/client/content/scenarios/scenario_roster_build.cpp`
- `Sunrise/src/client/content/scenarios/scenario_roster_groups.cpp`
- `Sunrise/src/client/content/scenarios/scenario_roster_publish.cpp`
- `Sunrise/src/client/content/scenarios/scenario_slot_classification.cpp`
- `Sunrise/src/state/build_data/scenarios/definition.h`
- `Sunrise/src/state/build_data/scenarios/scenario_catalog.cpp`
- `Sunrise/src/middleware/content/packages/tables/scenario_reader.cpp`
- `Sunrise/src/middleware/content/packages/tables/scenario_walk.cpp`

54.9 Spawn sets

- `Sunrise/src/client/content/spawn_sets/`

- spawn_set_build.cpp
- Sunrise/src/client/content/spawn_sets/
spawn_set_catalog_builder.cpp
- Sunrise/src/state/build_data/spawn_sets/
definition.h
- Sunrise/src/state/build_data/spawn_sets/
spawn_set_catalog.cpp

54.10 Auth and sense data

- Sunrise/src/middleware/bap/activity_message/
sensor_auth_update.h
- Sunrise/src/middleware/bap/activity_message/
activity_sensor_auth_encoder.cpp
- Sunrise/src/middleware/bap/activity_message/
activity_sensor_auth_blocks.cpp
- Sunrise/src/middleware/bap/activity_message/
activity_sensor_auth_bodies.cpp
- Sunrise/src/middleware/bap/activity_message/
activity_sense_update_parser.cpp
- Sunrise/src/server/bap/encrypted/push/activity/
activity_roster_push.cpp
- Sunrise/src/server/bap/encrypted/
activity_message/patch_epoch/
activity_patch_epoch_route.cpp

54.11 World and client hooks

- Sunrise/src/client/hooks/bootflow/spawn_hold.cpp
- Sunrise/src/client/hooks/network/
bubble_authority/
bubble_authority_replacements.cpp
- Sunrise/src/client/hooks/network/
bubble_authority/scope/bubble_authority_scope.cpp
- Sunrise/src/core/settings/client/definition.h
- Sunrise/src/core/settings/client/
client_settings_parser.cpp

54.12 Case evidence documents

The general handbook uses evidence from these detailed records:

- C:\Destiny 2 Development\HOMECOMING-FINDINGS.md
- C:\Destiny 2 Development\HOMECOMING-HANDOFF.md
- C:\Destiny 2 Development\HOMECOMING-MATCHMAKING-HANDOFF.md
- C:\Destiny 2 Development\HOMECOMING-QOS-HANDOFF.md
- C:\Destiny 2 Development\HOMECOMING-AH-HANDSHAKE-HANDOFF.md
- C:\Destiny 2 Development\HOMECOMING-KIND22-HANDOFF.md
- C:\Destiny 2 Development\HOMECOMING-AUTHORED-ROUTE-HANDOFF-20260819.md
- C:\Destiny 2 Development\HOMECOMING-PHASE5-BUILDOUT.md
- C:\Destiny 2 Development\SUNRISE-HOMECOMING-BRIEFING-FOR-CODEX.md
- C:\Destiny 2 Development\OMEGA-PURPLE-VFX-HANDOFF-20260822.md
- C:\Destiny 2 Development\OMEGA-MISSION-RECONSTRUCTION-COMPLETE-DOCUMENTATION-20260822.md

Use the case document when you need exact timestamps, handles, patches, or run conclusions.

Use this handbook when you need the general system model.

55. Runtime settings

Settings can select different native paths.

Record all relevant settings for every run.

55.1 Fade release

CURRENT SOURCE: `fadeRelease` defaults to true.

It releases the world-transition fade channel at the in-world step.

This setting covers a spawn path that never performs the normal release.

55.2 Join readiness

CURRENT SOURCE: `forceJoinRequestReady` defaults to true.

It forces the observed status 5-to-6 readiness check.

Two terms in that check are local client flags that no known host message changes.

Treat this setting as a client bridge.

55.3 Region privacy

CURRENT SOURCE: `regionPrivate` defaults to false.

False permits the public activity-host route.

True reports a public region as private and loads it solo.

A forced destination can still load solo.

55.4 Replicated participation record

CURRENT SOURCE: `pinReplicatedRecord` defaults to true.

It selects the replicated participation record instead of the inactive local copy.

The inactive copy has a spawn-gate byte that the recovered wire fields do not reach.

55.5 Spawn hold

CURRENT SOURCE: `holdSpawn` defaults to true.

CURRENT SOURCE: `spawnHoldMs` defaults to 30,000 milliseconds.

55.6 Authored roster exception

CURRENT SOURCE: `rosterForceAuthored` defaults to false.

It adds exact Omega opening objects during build-data generation.

Do not enable it globally without reviewing the selected scenario.

55.7 Authored mission seed

CURRENT SOURCE: `seedAuthoredSensors` defaults to false.

It publishes recovered neutral mission components after the client reaches the world.

55.8 Diagnostic core slot types

The current core diagnostic set emphasizes these slot types:

- 13: participation
- 16: package state
- 17: activity lifetime
- 18: activity script
- 35: mission director
- 41: queues

This set identifies foundational state.

It is not a complete mission roster.

56. Handoff requirements

A handoff must let another investigator continue without repeating the same captures.

56.1 Required context

Include these items:

- Date and time zone
- Source branch and revision
- Dirty-tree summary
- Executable hash
- Built DLL hash
- Installed DLL hash
- Active settings
- Destination
- Expected result
- Actual result

56.2 Required evidence classification

Label every conclusion as `CONFIRMED`, `CURRENT SOURCE`, `HISTORICAL`, `INFERRED`, `HYPOTHESIS`, `DISPROVEN`, or `UNKNOWN`.

Do not use `probably fixed` as an evidence label.

56.3 Required layer boundary

State the first successful layer.

State the first failed layer.

Example:

Phase 1 named the registry. The runtime factory created no object.

The failure is between registry publication and runtime construction.

56.4 Required negative evidence

List every route that a bounded test disproved.

Include the predicted result and actual result.

This list prevents an investigation loop.

56.5 Required next test

Describe the next test with one changed input and one stop condition.

Do not provide a list of several speculative fixes as one next step.

56.6 Required rollback

Identify the code, setting, or hook that the next investigator must remove after the test.

56.7 Exact versus local names

Mark recorder-local names clearly.

Examples include `Scene 1`, `Candidate 8`, `identity2`, `BC30`, and `AAD0`.

Do not present a recorder-local label as a universal engine term.

57. Definition of a reconstructed mission

A mission is not reconstructed when only its map loads.

Use the following completion levels.

57.1 Level 1: destination complete

The client loads the correct package, bubble, slice, and spawn.

57.2 Level 2: session complete

The client has stable group membership, activity membership, and activity-host state.

57.3 Level 3: authority complete

The required roster groups, runtime objects, and auth bodies exist.

57.4 Level 4: presentation complete

Required scenes, actors, models, dialogue, music, and effects run in the correct order.

57.5 Level 5: gameplay complete

Required collision, triggers, devices, enemies, and objectives function.

57.6 Level 6: lifecycle complete

Each object initializes, transitions, retires, and releases correctly.

57.7 Level 7: progression complete

Mission completion advances the activity and updates objective, reward, and quest state where required.

57.8 Level 8: repeatability complete

The route works after restart, on repeated entry, and after the expected failure or reconnect paths.

57.9 Claim rule

State the achieved level in every status report.

Do not use the word `complete` without its level.

58. Glossary

Use these terms consistently.

Activity host

The service role that owns activity membership, entity slots, bubble authority, and mission-state publication.

Activity-host identifier

A nonzero identifier that selects one activity-host endpoint.

It is not the activity-session identifier.

Activity message

A message inside the service-8 or service-9 activity-host envelope.

Activity script

The recovered type-18 authority object that holds activity-script state.

It is one part of the mission system.

Activity session

A process-owned record that binds a destination, client member, slots, membership, and bubble grants.

Advertisement

A matchmaking record that publishes a join descriptor and regional host identity.

Ambassador

The activity member that represents an activity host to a joining citizen.

The ambassador slot can differ from the local slot.

Auth data

Server-authoritative state for one roster object.

Type-5 phase 2 publishes this data when the slot permits auth state.

Authored event

A timeline event that creates or changes a scene output such as an effect.

BAP

The service protocol used for account, web, activity-host, matchmaking, and related messages.

Biased field

A field whose wire value includes a fixed offset from its semantic value.

Object type and index references use biases in the recovered auth protocol.

Bubble

One scenario region.

The activity state supports 64 usable bubble indices.

Bubble authority

The grant state that permits an activity host to publish objects for one bubble.

Cast

The set of actors that a scene scheduler controls.

Client executor

A local hook or native call that makes the installed client perform an operation directly.

Commit

The transaction phase that validates expected revisions and applies a prepared state change.

Component

A native runtime subsystem attached to an object.

Examples include animation, presentation, effect, sensor, or collision components.

Composite

A parent runtime object that combines lower effect or presentation data for a final consumer.

CONFIRMED

An evidence label for a statement proved by source, decoded data, log, or controlled visual test.

CURRENT SOURCE

An evidence label for behavior present in the current worktree.

It does not prove installed runtime behavior.

Descriptor

An exact encoded object that carries selection or join data.

Preserve unknown descriptor bytes or bits.

Destination

The selected package, activity, bubble, slice, spawn, and descriptor state for one activity session.

DTLS

Datagram Transport Layer Security.

It protects the observed gameplay transport after association.

DISPROVEN

An evidence label for a hypothesis that a controlled test rejected.

Entity slot

One authority index in the 8,192-slot activity space.

Evidence boundary

The point between the last proved layer and the first unproved layer.

Forced destination

An operator-selected process-local destination that replaces the client selection.

FNV-1

The hash algorithm used for recovered spawn-set name hashes.

Generation

A value that distinguishes a new lifetime of a reused handle, object, or roster set.

Gameplay group

The peer session that publishes members, players, parameters, and host transitions.

Group host

The server role that owns the gameplay group session.

Group parameter

One of the 25 indexed shared values in the gameplay group-session registry.

Handle

A runtime object identity that usually includes an index and generation.

HISTORICAL

An evidence label for behavior validated in an earlier branch, build, or run.

Host simulation

The path where Sunrise publishes service and authority state for native client systems to consume.

HYPOTHESIS

An evidence label for a testable proposal that evidence has not proved.

INFERRED

An evidence label for the best semantic explanation supported by indirect evidence.

Join descriptor

An exact 128-byte matchmaking and peer-join descriptor.

Lifecycle edge

A state boundary that performs native work.

Examples include creation, activation, handoff, retirement, and completion.

Machine identifier

The gameplay identity for one client or advertised peer machine.

Manager

A native object that owns a state machine or group of runtime objects.

Name the exact manager in a technical note.

Member key

The identity that binds an activity membership row to its joined client.

Membership

The complete state that identifies participants, their regions, and their connection roles.

Activity membership and gameplay-group membership are separate systems.

Mission director

The recovered type-35 authority object that coordinates mission-director

state.

Mission graph

The complete authored set of conditions, states, transitions, and outputs for a mission.

Sunrise does not yet have a complete general interpreter for this graph.

Native client

The installed retail game executable and its internal systems.

NAT

Network Address Translation.

The introduction exchange helps two advertised endpoints establish a usable network path.

Native runtime object

An in-memory client object that a native factory constructs from an authored definition.

Neutral body

A complete schema-valid object body that does not request an active mission edge.

Neutral does not mean zero-filled.

Online-session identifier

The identifier in an advertised citizen record that names the ambassador activity host in the recovered path.

Patch epoch

A connection-owned value that the client sends and type-5 phase 2 must echo.

Phase 1

The type-5 stage that registers roster groups, slots, and bubble ownership.

Phase 2

The type-5 stage that applies auth and sense object bodies.

Postcondition

The state in the owning object after a call or message completes.

Prepare

The transaction phase that validates input and creates a value-only pending plan.

Presentation

The native systems that create visible models, animation output, dialogue, music, and effects.

Provider

The runtime source that supplies a transform or another value to an effect or presentation object.

QoS

Quality of Service.

The recovered probe checks endpoint suitability and carries a structured response payload.

Registry

An authored collection that maps group and slot identities to object definitions.

Registry mount

The native lifecycle operation that makes a registry available to an active authority container.

Reading a registry from a package is not a mount.

Renderer input

The final data that the rendering system consumes.

A lower object field is not necessarily a renderer input.

Revision

A monotonic value that orders committed snapshots or validates a pending transaction.

RVA

Relative virtual address.

It identifies a location relative to one executable module base and is build-specific.

Roster

The selected groups and descriptor-backed object slots for an activity scenario.

Runtime object count

The number of constructed objects in a specified native authority container.

Use it to distinguish registration from construction.

Scenario

The content definition that organizes bubbles, slices, packages, roster groups, and spawn data for an activity.

Scene

A native timeline system that can own casts, animation, events, effects, and lifecycle reports.

Scene authority

The mission object that permits or selects a scene lifecycle.

Scene selector

The state that selects or activates one authored scene entry.

Sense data

Observed client or object state that the mission host can use as an input.

The client sends important sense changes with activity-message type 6.

Session identifier

A generic phrase that is unsafe without a qualifier.

Use activity-session, activity-host, group-session, online-session, or advertisement identifier.

Slice

One authored state selection inside a bubble or scenario region.

SOID

The service object identifier used by account, character, and participation data.

Spawn set

A named and hashed collection of authored arrival points for a map and package context.

Stable owner

The earliest object whose identity and lifetime remain valid across the investigated operation.

State sequence

The type-5 roster generation value.

It is not an ordinary packet revision.

Transaction

A prepared and validated state change that commits as one unit.

Transform bank

An authored indexed set of transforms or transform sources for a scene or effect event.

Transform provider

The runtime object that resolves the selected root, marker, socket, or fixed transform.

Transition token

An activity membership value that signals a new activity load or transition generation.

UNKNOWN

An evidence label for a question that the available evidence cannot answer.

VFX

Visual effects.

This handbook uses the full term where practical.

World device

A mission-owned environmental object such as a wall, portal, switch, or barrier.

Its visual, collision, and authority parts can be separate.

59. Useful log queries

Use `rg` for focused log review.

Set the log path once in PowerShell:

```
$sunriseLog = 'C:\Destiny 2 Development\bin\x64\Sunrise\l
```

59.1 Process and build identity

```
rg -n "process|build|hash|image|module|startup" $sunriseL
```

Confirm the process boundary before all other queries.

59.2 Destination and activity session

```
rg -n "service.?6|service.?7|destination|activity.?sessio
```

59.3 Matchmaking and advertisement

```
rg -n "matchmaking|advertisement|search|locate|configurat
```


59.4 Gameplay group

```
rg -n "peer|application.?ready|membership|parameter|activ
```

59.5 Activity messages

```
rg -n "activity.?message|msg.?type|patch.?epoch|entity.?s
```

59.6 Roster and runtime objects

```
rg -n "roster|sensor.?auth|phase.?1|phase.?2|runtime.?obj
```

59.7 Mission and sensors

```
rg -n "mission|director|activity.?script|sense|trigger|se
```

59.8 Scene and actor lifecycle

```
rg -n "scene|cast|actor|entity.?factory|model|presentatio
```

59.9 Effect provenance

```
rg -n "effect|vfx|provider|transform.?bank|composite|rend
```

59.10 Failure-only view

```
rg -n -i "fail|reject|invalid|mismatch|absent|timeout|dro
```

Do not interpret an isolated matched line without its activity and connection context.

Use a time window or correlation key to collect the related records.

60. Non-regression requirements

A new mission feature must not break the general session and world path.

Run the applicable checks after a significant change.

60.1 Bootstrap checks

- The secure BAP channel starts.
- Every request with a defined response receives one.
- Echo and subscriber registration remain functional.
- No pending-request ring grows without a terminal response.

60.2 Destination checks

- A normal client destination still works without a forced override.
- A forced destination applies only when configured.
- The raw selection descriptor remains intact.
- Activity-session identifiers remain nonzero and unique.

60.3 World checks

- The player reaches the correct bubble and slice.
- Spawn hold releases after Arrived.
- The fade channel releases.
- Reentry does not reuse a stale transition token.

60.4 Gameplay checks

- Peer application-ready still occurs.
- The self-member row resolves.
- Membership revisions increase.
- The `activityHost` parameter remains nonzero.
- Reliable publication does not fill its queue permanently.

60.5 Activity-host checks

- State refresh preserves the type-1, type-12, and roster order.
- Patch epoch remains connection-owned.
- Entity-slot masks remain disjoint.
- Bubble grants update after a region move.
- Roster generation remains stable after warmup.

60.6 Mission checks

- Neutral mission objects still initialize.
- A repeated sensor packet does not restart a one-shot stage.
- A scene starts only from its intended edge.
- Mission completion does not depend on a diagnostic timer.
- Retained presentation does not block logical completion.

60.7 Presentation checks

- Exact model filters do not hide a correct cast from another scene.
- Effect filters use full definitions and generations.
- Diagnostic offsets are removed.
- High-volume recorders are disabled for normal use.

60.8 State checks

- Failed encoding does not commit state.
- Failed send does not advance publication stage.
- A stale pending transaction does not commit.
- Capacity exhaustion produces a bounded log.
- Connection teardown clears connection-owned state only.

60.9 Repeatability checks

Run these cases when the changed layer can affect them:

1. Fresh process and first entry
2. Activity restart in the same process
3. Return to orbit and reentry
4. Region transition
5. Gameplay reconnect
6. Normal scene completion
7. Mission failure or reset

Record the completion level for each case.

61. Compact system truth

The client already contains much of the mission content.

Sunrise must supply the correct service state, authority, and lifecycle inputs.

A successful lower layer does not prove the next layer.

Preserve exact opaque data when its schema is incomplete.

Create schema-valid neutral authority objects before active edges.

Use real sensor and lifecycle events instead of guessed timers.

Trace scenes, actors, models, and effects as separate owners.

Trace an effect to the final renderer input before you change coordinates.

Stop packet-body work when the native runtime object does not exist.

Treat presentation, gameplay, lifecycle, and progression as separate completion levels.

Use bounded tests with explicit stopping points.

Archive negative evidence.

Do not repeat a disproven route without new evidence.